

GovAsset Platform — Complete Presentation Guide

Read everything below and you will understand your system fully. All answers use simple language — no unnecessary technical words.

Table of Contents

#	Part	What it explains
1	Django Request Flow	Step-by-step: what happens from typing a URL to getting a response
2	Multi-tenancy	What multi-tenancy means (apartment analogy)
3	django-tenants	How django-tenants creates separate schemas per ministry, with code evidence
4	Keycloak SSO Flow	The 14-step login flow + Keycloak purpose + Django vs Keycloak relationship
5	Keycloak vs SimpleJWT	When each is used, why mobile doesn't use Keycloak, why web doesn't use JWT
6	0.0.0.0:8000 Explained	Why we use it, what happens if we don't
7	All Code Components	Decorators, serializers, middleware, permissions, exception handler, pagination explained
8	Security Features	All 13 security features with file/line evidence
9	Database Design	Public schema vs tenant schemas, why not one big table
10	Other Technologies	DRF, CORS, mozilla-django-oidc, drf-yasg, SimpleJWT, decouple, etc.
11	Deployment to Cloud	What you need for production, do you use 0.0.0.0?
12	JWT Tokens	Access vs refresh token, 3 parts of JWT, expiration, how to change
13	Integration With Other Groups	How they call our API, what we share, example code, all endpoints listed
14	Why Each Ministry Needs a Domain	Why not just use /ministry/1/ in the URL
15	Presentation Tips	Common panel questions, demo sequence, how to answer
16	Quick Reference	Every key file and what it does
17	Complete Request-to-Response Flow	Web browser flow + mobile API flow in full detail
18	Glossary	Simple definitions of every technical term

#	Part	What it explains
19	Remember This for the Panel	Key sentences to memorize
20	CSS Quick Reference	How to change colors, text size, backgrounds — inline CSS & Bootstrap Icons
21	Database Deep Dive	All tables, columns, why shared vs private schemas, why PostgreSQL auto-starts
22	Making the System Live	Production setup, domains, code changes, how other groups access in production
23	Dev Setup: Web + Mobile	How two developers work together, same WiFi, different laptops
24	Complete File-by-File Explanation	Every folder and every <code>.py</code> file explained, with key classes and code evidence
25	Important Commands & Setup Reference	Full step-by-step: PostgreSQL, find IP, start Keycloak, start Django, connect phone, register domain, Flutter rebuild, demo data, troubleshooting, test accounts

PART 1: DJANGO REQUEST FLOW — What happens from URL to response?

In this part:

- [11-step request flow](#)
- [Our project's specific flow \(Keycloak\)](#)

11-step request flow

When someone types a URL or clicks a link, here is the exact path:

Browser types: `http://moh.localhost:8000/assets/`

Step 1: Browser sends the URL to your computer (localhost)

Step 2: Django receives it and looks at the URL path: `/assets/`

Step 3: django-tenants middleware checks: "What domain is this?" → `moh.localhost`

Step 4: django-tenants looks up "moh" in the Domain table → finds Ministry of Health

Step 5: django-tenants switches the database to `moh_schema`

Step 6: Django matches `/assets/` against `urlpatterns` in `urls.py`

Step 7: Finds `asset_list_view` (or `AssetListCreateAPIView` if API)

Step 8: The view function runs – it queries the database

Step 9: The database only sees `moh_schema`'s tables (data isolation!)

Step 10: The view returns an HTML page or JSON response

Step 11: Django sends the response back to the browser

Key point: django-tenants steps 3-5 happen on EVERY request, BEFORE your view code runs. This is why `TenantMainMiddleware` must be the first middleware in the list.

Our project's specific flow:

```
User → Login Page → Clicks "Sign in with Government SSO"
→ Django redirects to Keycloak login page (separate website)
→ User types username/password on Keycloak
→ Keycloak redirects back to Django with a code
→ Django exchanges code for tokens (server talks to server)
→ Django verifies the token signature (proves it's really Keycloak)
→ Django finds or creates the user in its database
→ User is redirected to /dashboard/
```

PART 2: MULTI-TENANCY — What does it mean?

In this part:

- [What multi-tenancy means](#)
- [Wrong approach \(ministry_id column\)](#)
- [Our approach \(PostgreSQL schemas\)](#)

What multi-tenancy means

Multi-tenancy means one system serves many customers, and each customer's data is completely separate.

Think of an apartment building:

- The building = one Django application
- Each apartment = one ministry's data
- Each apartment has its own locked door (PostgreSQL schema)
- A person in apartment 3 cannot walk into apartment 5

Our system: Instead of apartments, we have ministries (MOH, MOF, etc.). Each ministry gets their own locked room in the database. Ministry of Health staff ONLY see Health data. Ministry of Finance staff ONLY see Finance data.

NOT multi-tenant (wrong approach):

```
One big table with a "ministry_id" column
→ Risk: A bug might accidentally show Finance data to Health users
→ Risk: A programmer forgets to filter by ministry_id
```

OUR approach (correct):

Separate PostgreSQL schemas

- Each ministry has its own table called `assets_asset`
- If a query doesn't filter by ministry, it STILL only sees one schema
- Data isolation is ENFORCED BY THE DATABASE, not by our code

PART 3: django-tenants — How exactly does it work?

In this part:

- [How django-tenants works \(3 key mechanisms\)](#)
- [Evidence in our code](#)

How django-tenants works

django-tenants is a library that:

1. **Changes the database engine** — Instead of the normal PostgreSQL driver, it uses `django_tenants.postgresql_backend`. This custom driver adds `SET search_path = moh_schema, public;` before EVERY query.
2. **Identifies the tenant from the URL** — `TenantMainMiddleware` reads the domain (e.g., `moh.localhost`), looks it up in the `tenants_domain` table, finds which Ministry it belongs to, and sets the schema name.
3. **Creates schemas automatically** — When a Super Admin creates a new Ministry, django-tenants runs `CREATE SCHEMA moh_schema` in PostgreSQL and creates all the tables inside it (`assets_asset`, `organizations_orgunit`, etc.).

Evidence in our code:

settings.py:

```
# Custom PostgreSQL backend (line 121)
DATABASES = {
    'default': {
        'ENGINE': 'django_tenants.postgresql_backend', # ← This is key
    }
}

# Router tells Django to use this backend for all apps (line 130)
DATABASE_ROUTERS = ('django_tenants.routers.TenantSyncRouter',)

# Which apps are shared (one copy, public schema)
SHARED_APPS = ['django_tenants', 'tenants', 'authentication', ...]

# Which apps are per-tenant (separate copy per ministry)
TENANT_APPS = ['organizations', 'assets']
```

tenants/models.py:

```
class Ministry(TenantMixin):
    # TenantMixin gives us schema_name field automatically
    auto_create_schema = True # When saved, PostgreSQL creates a new schema
```

Every view that queries tenant data:

```
from django_tenants.utils import schema_context

# Switch to the ministry's private schema
with schema_context(user.ministry_schema):
    # This query ONLY sees that ministry's data
    assets = Asset.objects.all()
```

PART 4: KEYCLOAK SSO — The full login flow (simple version)

Keycloak is a separate program that handles logging in. Django does NOT handle passwords — Keycloak does.

The flow with 3 characters:

```
Character 1: THE USER (opens browser)
Character 2: DJANGO (our app)
Character 3: KEYCLOAK (the identity server)
```

Step 1: User visits `http://moh.localhost:8000/login/`
Step 2: Django shows the login page with a "Sign in with Government SSO" button
Step 3: User clicks the button → browser goes to `/oidc/authenticate/`
Step 4: Django says "I don't handle passwords. Go to Keycloak."
 Redirects to: `http://localhost:8180/realms/govasset/protocol/openid-connect/auth`
Step 5: Keycloak shows its own login page (looks different from Django)
Step 6: User types username and password
Step 7: Keycloak checks: "Is this password correct?" If yes, continues
Step 8: Keycloak creates a temporary CODE and sends the browser back to Django
 URL: `/oidc/callback/?code=xyz123`
Step 9: Django takes the code and calls Keycloak secretly (server-to-server):
 "Hey Keycloak, here's the code. Give me the real tokens."
Step 10: Keycloak verifies the code and sends back: `access_token + id_token`
Step 11: Django checks the `id_token`'s SIGNATURE (like checking a passport seal)
 - It fetches Keycloak's public key
 - Verifies the signature matches (proves Keycloak signed it)
 - Checks expiration date
Step 12: If everything checks out, Django looks up the user in its database

Step 13: If user exists → login successful → redirect to /dashboard/
Step 14: If user doesn't exist → create PendingAccess → show error message

How Django knows the user is real (not a hacker):

Three checks, like a passport check at the airport:

Check 1 – THE SIGNATURE (cryptographic):
Keycloak signs the token with its PRIVATE key (like a government stamp)
Django verifies with Keycloak's PUBLIC key (like checking the stamp is real)
If someone forged a token, the signature won't match → REJECTED

Check 2 – THE PASSPORT DETAILS (claims):
Django checks:
- Is the token expired? (exp field)
- Was this token meant for OUR app? (aud field)
- Did Keycloak really issue this? (iss field)
If any fails → REJECTED

Check 3 – THE DATABASE LOOKUP (application):
Django looks in its own database: "Is this user registered?"
If no matching user found → REJECTED (shows pending access message)

What is the real purpose of Keycloak? (Why do we have it at all?)

This is the most important question. Here is the answer:

Keycloak exists so that one username + password works across MULTIPLE government systems, not just ours.

Imagine the government has 20 different systems:

- Our asset management system
- A human resources system
- A budget system
- A procurement system
- A document management system

Without Keycloak, a user would need **20 different usernames and passwords** — one for each system. With Keycloak, they log in ONCE and access ALL 20 systems.

Analogy: Keycloak is like a government ID card. The ID card is issued by the central government (Keycloak). It works at the health ministry (our system), the finance ministry (another system), the education ministry (another system). You don't need a separate ID card for each ministry.

OUR APPROACH (with Keycloak):

User has ONE account in Keycloak

```
Our system:      "Is this user real? Let me check
                  Keycloak's signature on their token"

HR system:       "Same user? Let me check Keycloak
                  too. Yes, same person."

Budget system:   "Same user. Same ID. Same credentials."
```

ALTERNATIVE (without Keycloak):

```
User has SEPARATE accounts in every system

Our system:      username: moh_admin, password: abc123
HR system:       username: aminah,   password: xyz789
Budget system:   username: ahasan,   password: qwe456

User forgets 3 passwords. Writes them on sticky notes.
Sticky note falls off. Security risk.
```

Key point: Keycloak is NOT just for us. It is the CENTRAL identity system for ALL government online services. Our Django app is just ONE of many systems that use it.

How Django and Keycloak work together — the relationship

KEYCLOAK'S JOB:

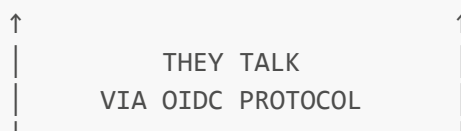
```
Store passwords
Check passwords
Issue tokens
Handle "forgot pwd"
Handle "new user"

Users from ALL
government systems
live here
```

DJANGO'S JOB:

```
Store user profiles
Store roles
Store ministry
Store permissions
Store audit logs
Handle API tokens

Only our system's
users live here
```



EXAMPLE: User "amina@moh.go.tz"

In Keycloak: username + password (the "who are you?" part)

In Django: role = MINISTRY_ADMIN, ministry = MOH (the "what can you do?" part)

Why both? Why not just Django or just Keycloak?

If we ONLY used Keycloak: Keycloak doesn't know what ministry the user belongs to, what their role is, what assets they manage. Keycloak only knows "this username, this password, this email."

If we ONLY used Django: Each government system would have its own login. User needs 20 accounts. No centralized identity.

Does Django also store user credentials (passwords)?

The answer depends on how the user was created:

For web-only users (logged in via Keycloak): NO. Django never sees the password. The user types their password on Keycloak's page, not Django's page. Django receives a token FROM Keycloak that says "this user is authenticated," and stores an **unusable password** in its database — a placeholder like `!aBcDeF12345`.

This means:

- The user CAN log in through the web browser (Keycloak handles password checking)
- The user CANNOT log in through the mobile API (no valid Django password to check against)

For mobile/API users (logged in via direct API): YES. Django receives the password from the mobile app and checks it against its database. The password is stored securely (hashed using Django's password hasher).

This means:

- The user CAN log in through the mobile API (Django checks the stored password)
- The user can ALSO log in through the web (if they also have a Keycloak account)

Key insight — A user needs TWO different credentials for web + mobile:

EXAMPLE: User "amina@moh.go.tz"

KEYCLOAK PASSWORD
(for web browser)

password: "M0H@2024"

She types on Keycloak
login page

Stored in Keycloak's
own database

DJANGO PASSWORD
(for mobile app)

password: "Am!naM0bile99"

She types in Flutter app
login screen

Stored in Django's database
(hashed, never in plain text)

These are **SEPARATE** passwords stored in **SEPARATE** databases. Changing one does not change the other.

How does a user get a Django password?

1. When a Super Admin creates the user directly in Django (via the user management page), they set a password
2. When a PendingAccess request is approved, the admin sets a password at that time

3. The password is sent to Django, hashed, and stored in the `password` field of `authentication_customuser` table

What happens if a Keycloak-only user tries to log in via the mobile app? They fail. Django has no valid stored password for them (only an unusable placeholder). The mobile app shows "Invalid credentials." This is correct behavior — the admin must set a separate Django password if the user needs mobile access.

What if a user exists in Keycloak but NOT in our Django database?

Step 1: User goes to our login page, clicks SSO, types their Keycloak username/password correctly. Keycloak says "Yes, this person is real."

Step 2: Keycloak sends the user back to Django with a valid token. Django checks the token and verifies: "Yes, Keycloak really signed this. This user is real."

Step 3: Django looks in its OWN database for this user. If the user exists → login success. If the user does NOT exist:

- Django creates a PendingAccess record
- Django shows the user a page saying:
"Your account is not yet registered. Contact your administrator."
- The Super Admin sees a pending approval request
- The Super Admin can approve or reject
- If approved: Django creates the actual user in its database
(with role, ministry, and a Django password for mobile access)
- The user can now log in via web (Keycloak) AND mobile (Django password)

Code evidence (authentication/oidc_backend.py):

```
def create_user(self, request, user_claims):
    # This runs when Keycloak says "user is real"
    # but Django has no matching user
    ...
    PendingAccess.objects.create(
        email=email,
        keycloak_id=sub,
        first_name=first_name,
        last_name=last_name,
    )
    return None # ← Returning None means "user not allowed yet"
```

How does someone get created in Keycloak alone? (Without Django knowing)

An IT officer at a ministry can create a user in Keycloak directly without involving Django at all:

```
IT Officer goes to http://localhost:8180 → govasset realm → Users → Add User
→ Sets username: "amina", email: "amina@moh.go.tz"
```

- Sets a temporary password
- Tells Amina: "Your account is ready. Go to the system and log in."

↓

Amina opens our web app → Clicks "Sign in with Government SSO"

- Keycloak says: "Yes, I know you. Password is correct."
- Keycloak sends Amina to Django with a valid token
- Django checks its database... "I don't know this person."
- PendingAccess record is created
- Amina sees: "Your account is awaiting approval. Contact your administrator."
- Super Admin sees the pending request → approves it
- Super Admin assigns Amina's role and ministry
- Super Admin sets Amina's Django password (for mobile access)
- Amina can now log in via web AND mobile

Important: Keycloak alone is NOT enough to access our system. The user must also exist in Django with a role and ministry assignment. PendingAccess is the bridge between "Keycloak knows you" (identity) and "Django knows you" (authorization).

Can someone be created in Django alone? (Without Keycloak)

YES. A Super Admin can create a user directly in Django using the user management page. This creates the user only in Django. The user can then log in via the mobile app (API login with Django password) but NOT via the web (no Keycloak account).

If the user also needs web access, the Super Admin must ALSO create them in Keycloak. Our code in [authentication/keycloak_admin.py](#) helps automate this — when you create a user in Django, it can optionally create a matching account in Keycloak too.

Summary: Two creation paths

PATH 1: Keycloak → Django

User created in Keycloak by IT officer

- User tries web login
- PendingAccess created
- Super Admin approves
- Django user created with unusable password
- User can log in via web ONLY
- (Mobile access requires admin to set separate Django password)

PATH 2: Django → Keycloak

Super Admin creates user in Django

- Django creates matching account in Keycloak automatically
- User gets BOTH:
 - Django password (for mobile API login)
 - Keycloak account (for web browser SSO)
- User can log in via web AND mobile

The PendingAccess flow (Path 1) is designed for the real-world scenario where an IT officer creates users in the central identity system (Keycloak) without coordinating with our team. PendingAccess ensures no user gets

access to our system until a Super Admin explicitly approves them and assigns a role.

Does Django manage sessions?

YES. Django manages TWO kinds of sessions:

Kind	What it is	Where it's stored
Django session cookie	"This browser is logged in as user X"	<code>django_session</code> table (public schema)
Keycloak session	"This user has an active Keycloak login"	Inside Keycloak's database

When you log in via the web:

1. Keycloak creates a session (you are logged into Keycloak)
2. Django creates a session (you are logged into Django)
3. Both are active at the same time

When you log out:

1. Django destroys its session (you're logged out of Django)
2. Django tells Keycloak to destroy its session too (you're logged out of Keycloak)
3. Both sessions are gone

Forgot password — How it works with Keycloak

Question: Does the "forgot password" feature still work now that Keycloak handles passwords?

Answer: Yes, but it works differently — and this is actually more secure.

Before Keycloak: Django's own "forgot password" flow would send an email with a reset link, and the user would reset the password in Django's database.

After Keycloak: The password doesn't live in Django's database anymore. Django's password field is empty (set to an unusable value) for Keycloak-authenticated users. The password lives entirely in Keycloak.

So "forgot password" now goes through Keycloak:

```
User clicks "Forgot Password" on Keycloak's login page
↓
Keycloak sends a reset link to their email
↓
User clicks the link, sets a new password in Keycloak
↓
Done – they can log in again with the new password
```

Django is not involved in this process. This is **more secure** because password reset logic is handled by a dedicated, tested security tool (Keycloak) rather than our own code.

Does our Django "forgot password" URL still work? It might appear to work (send an email), but it would reset a password that isn't actually used anymore. If your login page has a "forgot password" link that goes to Django's own reset page, you should remove that link or redirect it to Keycloak's login page where the real forgot-password button lives.

Wrong password logout — Double protection

Question: If someone types the wrong password 5 times, does the logout still work with Keycloak?

Answer: You actually have TWO separate logout systems protecting you:

Logout System 1 — Keycloak's own logout (for web browser users):

Keycloak itself watches for wrong passwords. After 5 wrong attempts, Keycloak locks that account for 15 minutes. This happens entirely inside Keycloak — Django doesn't even know about it.

```
Web user types wrong password 5 times on Keycloak's page
↓
Keycloak locks the account for 15 minutes
↓
Web user sees on Keycloak's page: "Account temporarily disabled"
↓
Django was never contacted – Keycloak handled everything
```

Logout System 2 — Django's own logout (for API/mobile users):

Your `authentication/models.py` has a `LoginAttempt` model that records every login attempt:

```
# authentication/models.py
class LoginAttempt(models.Model):
    username = models.CharField(max_length=150)
    ip_address = models.GenericIPAddressField()
    attempted_at = models.DateTimeField(auto_now_add=True)
    was_successful = models.BooleanField(default=False)
```

Your API login view checks this before allowing login:

```
# authentication/api_views.py (simplified)
class LoginAPIView(APIView):
    def post(self, request):
        username = request.data.get('username')
        ip = request.META.get('REMOTE_ADDR')

        # Count recent failed attempts from this IP
        recent_failures = LoginAttempt.objects.filter(
            username=username,
            was_successful=False,
            attempted_at__gte=timezone.now() - timedelta(minutes=15)
```

```

).count()

if recent_failures >= 5:
    return Response(
        {'error': 'Too many failed attempts. Try again in 15 minutes.'},
        status=429
    )

# Proceed with login
user = authenticate(username=username, password=password)

# Record this attempt
LoginAttempt.objects.create(
    username=username,
    ip_address=ip,
    was_successful=(user is not None)
)

```

Summary — Who protects whom:

Who is trying to log in	Which logout protects them
Web browser user (through Keycloak page)	Keycloak's own built-in logout
Mobile app user (through your API)	Your Django <code>LoginAttempt</code> logout
Another group's system (through your API)	Your Django <code>LoginAttempt</code> logout

You have double protection. Both paths are covered.

PART 5: What is the difference between Keycloak and SimpleJWT?

People get confused. Here is the simple answer:

	Keycloak	SimpleJWT
Used for	Logging in (web browser)	API calls (mobile app)
Who handles it	Keycloak server (separate program)	Django itself
What it produces	A Django session (cookie in browser)	A JWT token (text string)
When is it used	When user clicks "Sign in"	When mobile app calls API
Password involved?	Yes, user types password on Keycloak	Yes, on first login only

Think of it this way:

- **Keycloak** = The security guard at the gate who checks your ID
- **SimpleJWT** = The visitor badge you get AFTER the guard lets you in

The mobile app (Flutter) does NOT use Keycloak. Instead:

1. Mobile app calls `POST /api/auth/login/` with username + password
2. Django checks the password against its database (same as Keycloak)
3. Django returns a JWT token (valid 30 minutes)
4. Mobile app sends this token with every API call
5. When token expires, mobile app calls `POST /api/auth/refresh/`

Why doesn't the mobile app use Keycloak?

Keycloak uses a **browser redirect flow** — it sends the user to a Keycloak login page, the user types their password there, then it redirects back to Django.

This does NOT work on a mobile app because:

WEB BROWSER FLOW (works with Keycloak):

```
Browser → Django → redirect to Keycloak → user types password → redirect back
↑                                                         ↑
Browser can follow redirects automatically.
The "Sign in with Google" button you see on websites works this way.
```

MOBILE APP FLOW (cannot use Keycloak):

```
Flutter app → Django → "Go to Keycloak" → ???
```

↑

A mobile app is not a web browser. It cannot display Keycloak's login page. It cannot follow web redirects.

Option A: Open a web browser inside the app (WebView)

- Ugly, confusing, users don't trust it
- Keycloak session doesn't persist properly
- Many technical problems

Option B: Mobile app sends username + password directly to Django

- Simple, works every time
- Django checks the password and returns a JWT token
- This is what we do ✓

The real reason: Keycloak (and all OIDC providers) were designed for web browsers. Mobile apps use a different authentication pattern called "direct login" where the app sends credentials to an API and gets back a token. This is industry standard — even Google, Facebook, and Twitter have separate "mobile login" APIs that don't use browser redirects.

Why doesn't the web browser use SimpleJWT (like the mobile app)?

Three reasons:

Reason	Explanation
1. Security	With SimpleJWT, the user types their password directly into Django's login page. If someone steals the password, they have full access. With Keycloak, Django never sees the password — Keycloak handles it securely.

Reason	Explanation
2. Single Sign-On	If the web used SimpleJWT, users would need a separate login for EVERY government system (asset management, HR, budget, etc.) — 20 different passwords. With Keycloak SSO, one login works everywhere.
3. Government policy	The government mandates that ALL online services use Keycloak for authentication. We cannot build our own login page that bypasses Keycloak. This is a security policy, not a technical choice.

So why does the mobile app get an exception?

Because there is no other way. Mobile apps cannot use Keycloak's browser redirect flow (explained above). The government understands this and allows mobile apps to use direct API login. But the mobile app's login is STILL connected to Keycloak indirectly:

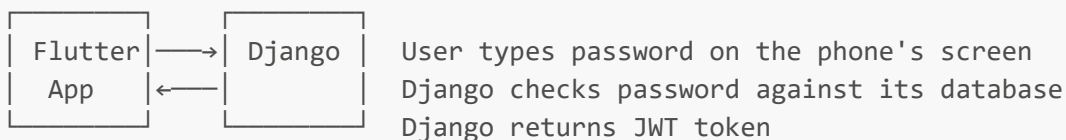
```
Mobile app login → Django checks password → Django stores users that
                                     came FROM Keycloak originally
                                     ↑
The user first had to exist in Keycloak and be approved in Django
via the web flow. The mobile app login is a CONVENIENCE for
already-approved users.
```

Summary: Two authentication paths side by side

PATH 1: WEB BROWSER (uses Keycloak)



PATH 2: MOBILE APP (uses SimpleJWT)



PART 6: Why 0.0.0.0:8000 instead of just **runserver**?

Normal runserver:

```
python manage.py runserver
→ Django listens ONLY on 127.0.0.1:8000
→ 127.0.0.1 = "this computer only"
```

- Your phone CANNOT reach this
- moh.localhost CANNOT reach this

0.0.0.0:8000:

- ```
python manage.py runserver 0.0.0.0:8000
```
- Django listens on ALL network interfaces
  - 0.0.0.0 = "everywhere"
  - Same WiFi network? Your phone CAN reach this
  - moh.localhost on same computer CAN reach this

### Analogy:

- 127.0.0.1 = a phone that only calls itself
- 0.0.0.0 = a phone that can receive calls from anyone

Do you need 0.0.0.0 in production (cloud)?

**NO.** In production, you use a proper web server like Nginx or Apache. The server listens on 0.0.0.0:80 (that's what web servers do by default). Django sits behind the web server and only listens on localhost (127.0.0.1:8000) — the web server forwards requests to it.

So: 0.0.0.0:8000 is ONLY for development when you need your phone or tablet to test.

What if you don't use 0.0.0.0?

Then your mobile phone connected to the same WiFi cannot access the website. You type `http://192.168.100.18:8000` on your phone and nothing happens, because Django only listens on 127.0.0.1 (which means itself only).

---

## PART 7: All Your Code Components — Explained Simply

What is `mozilla_django_oidc`?

It is a Django library that does the Keycloak handshake for us. Without it, we would have to write hundreds of lines of code to handle the OIDC protocol (redirects, code exchange, token verification). The library provides:

- `/oidc/authenticate/` — Redirects user to Keycloak login page
- `/oidc/callback/` — Handles the return from Keycloak, exchanges code for tokens
- `/oidc/logout/` — Logs user out of both Django and Keycloak

What is `rest_framework` (DRF)?

Django REST Framework is a library that makes it easy to build APIs (Application Programming Interfaces). An API is how the Flutter mobile app talks to our Django backend.

**Without DRF:** You would have to manually parse JSON from requests, manually convert database data to JSON, manually handle errors, etc.



**With DRF:** You define a serializer (a class that says "here are the fields") and a view (a class that says "what happens on GET, POST, etc."), and DRF handles everything.

What is **corsheaders**?

CORS = Cross-Origin Resource Sharing.

**Problem:** By default, a web browser on **flutterapp.com** cannot call an API on **moh.localhost:8000**. The browser blocks it for security.

**Solution:** CORS headers. We tell Django: "Hey, it's OK if the Flutter app calls our API from a different address."

Our settings:

```
CORS_ALLOW_ALL_ORIGINS = True # Allow Flutter from any address
CORS_ALLOW_CREDENTIALS = True # Allow cookies to be sent
```

What is **drf\_yasg**?

It automatically creates a Swagger documentation page at **/api/docs/**. This is a nice web page that shows ALL your API endpoints, with buttons to test them. Other groups (Group 2-10) use this page to understand how to call our API.

You don't write documentation — drf\_yasg reads your code and generates it.

What is **python-decouple**?

It reads secret values from a **.env** file. This keeps passwords out of your code.

```
Instead of writing the password in settings.py:
SECRET_KEY = 'my-secret-key-here' # BAD - visible in code

We do this:
SECRET_KEY = config('SECRET_KEY') # GOOD - read from .env file
```

Your **.env** file has:

```
SECRET_KEY=my-secret-key-here
DB_PASSWORD=my-db-password
```

The **.env** file is NEVER committed to GitHub (it's in **.gitignore**).

What is **psycopg2-binary**?

It is the driver that lets Python talk to PostgreSQL. Think of it like a translator — Django speaks Python, PostgreSQL speaks SQL, psycopg2 translates between them.

## What is Pillow?

A library that handles image files. When someone uploads a photo of an asset (like a laptop), Pillow processes the image — resizing, format conversion, etc.

## What are DECORATORS?

A decorator is a wrapper around a function. It runs code BEFORE your function runs.

**Analogy:** A security guard at the door of a building. Before you enter (before your function runs), the guard checks your ID.

```
@login_required_custom # ← This is a decorator (security guard)
@role_required('SUPER_ADMIN') # ← Another security guard
def ministry_list_view(request):
 # This code only runs if BOTH guards say OK
 ...
```

### Our custom decorators:

| Decorator                        | What it does                                     | Where it's used |
|----------------------------------|--------------------------------------------------|-----------------|
| @login_required_custom           | Blocks unauthenticated users → redirect to login | ALL views       |
| @role_required('MINISTRY_ADMIN') | Blocks wrong roles → redirect to dashboard       | Admin views     |
| @ministry_isolation_check        | Blocks users without a ministry schema           | Ministry views  |

### Code (authentication/decorators.py):

```
def login_required_custom(view_func):
 @wraps(view_func)
 def wrapper(request, *args, **kwargs):
 if not request.user.is_authenticated:
 messages.warning(request, "Please log in.")
 return redirect('login')
 return view_func(request, *args, **kwargs)
 return wrapper
```

## What are SERIALIZERS?

A serializer converts between two formats:

1. **Python objects** → **JSON** (for the API response)
2. **JSON** → **Python objects** (from the API request)

**Example:** When Flutter asks for an asset, Django:

1. Gets the Asset object from the database (Python object)
2. Passes it through AssetSerializer
3. DRF converts it to JSON automatically
4. Flutter receives: `{"id": 1, "name": "Dell Laptop", "asset_number": "MOH-ICT-0001"}`

### Our serializers (authentication/api\_serializers.py):

- `AssetSerializer` — Converts assets to JSON for the mobile app
- `AuditLogSerializer` — Converts audit logs
- `OrgUnitSerializer` — Converts organization units
- `UserProfileSerializer` — Converts user profiles (never exposes passwords!)
- `CustomTokenObtainPairSerializer` — Adds role, ministry\_schema to JWT tokens

### What is MIDDLEWARE?

Middleware is code that runs on EVERY request, BEFORE and AFTER your view.

Think of it as a pipeline:

```
Request comes in
→ Middleware 1 (TenantMainMiddleware: sets schema)
→ Middleware 2 (SecurityMiddleware: security headers)
→ Middleware 3 (SessionMiddleware: loads session)
→ Middleware 4 (CorsMiddleware: handles CORS)
→ Middleware 5 (AuthMiddleware: loads logged-in user)
→ Middleware 6 (SchemaMiddleware: OUR custom one)
→ YOUR VIEW FUNCTION
→ Back through all middlewares
Response goes out
```

### Our custom middleware (authentication/middleware.py):

```
class SchemaMiddleware(MiddlewareMixin):
 def process_request(self, request):
 # Skip public pages
 # For authenticated users, save their schema name on the request
 if hasattr(request, 'user') and request.user.is_authenticated:
 request.schema_name = request.user.ministry_schema or 'public'
 request.user_role = request.user.role
```

This means ANY view can access `request.schema_name` — no need to look it up from the database each time.

### What is the EXCEPTION HANDLER?

When an API error happens (e.g., user not authenticated, permission denied, not found), DRF returns errors in different formats. The Flutter app needs a CONSISTENT format.

Our custom handler (authentication/api\_exception\_handler.py) catches all errors and returns them in one shape:

```
{
 "error": True,
 "message": "Authentication credentials were not provided.",
 "code": "authentication_required",
 "status": 401
}
```

Flutter always knows: check `error`, read `message`, handle by `code`.

## What is PAGINATION?

When you have 1000 assets and the user views the list, you don't send all 1000 at once — you send 20 at a time (one page).

Our pagination helper (authentication/pagination.py):

```
def paginate_queryset(queryset, request, per_page=20):
 paginator = Paginator(queryset, per_page)
 page = paginator.page(request.GET.get('page', 1))
 return page, paginator
```

Usage in views:

```
page, paginator = paginate_queryset(assets, request, per_page=25)
page.object_list = the 25 items for this page
paginator.count = total number of items
paginator.num_pages = how many pages total
```

## What are PERMISSION CLASSES?

DRF has a built-in system for checking who can access API endpoints. We created custom permission classes:

**Where (authentication/api\_permissions.py):**

```
class HasMinistrySchema(BasePermission):
 """Block users who aren't assigned to any ministry."""
 def has_permission(self, request, view):
 if request.user.role == 'SUPER_ADMIN':
 return True
 return bool(request.user.ministry_schema)

class CanManageAssets(BasePermission):
 """Auditors can read only. Others can write."""
```

```
def has_permission(self, request, view):
 if request.method in ['GET', 'HEAD', 'OPTIONS']:
 return True # Anyone can read
 return request.user.role in ['SUPER_ADMIN', 'MINISTRY_ADMIN', ...]
```

### Usage in API views:

```
class AssetListCreateAPIView(APIView):
 permission_classes = [
 IsAuthenticated, # Must be logged in
 HasMinistrySchema, # Must have a ministry
 CanManageAssets, # Must have permission
]
```

---

## PART 8: SECURITY FEATURES

Here is EVERY security feature in the system and where to find it:

### 1. Database-level isolation

Users of one ministry CANNOT see another ministry's data — even if there is a bug in our code. PostgreSQL schemas enforce this.

- **Evidence:** `DATABASE_ENGINE = django_tenants.postgresql_backend`

### 2. Role-based access control

Each user has a role (SUPER\_ADMIN, MINISTRY\_ADMIN, AGENCY\_MANAGER, FACILITY\_CLERK, AUDITOR). Views check roles before running.

- **Evidence:** `@role_required('MINISTRY_ADMIN')` on every sensitive view

### 3. API permission classes

Same as roles but for API endpoints. Different levels: `IsSuperAdmin`, `IsMinistryAdmin`, `CanManageAssets`, `CanDeleteAssets`, `CanViewAuditLogs`.

- **Evidence:** `authentication/api_permissions.py`

### 4. Brute force protection

If someone tries to login 5 times with wrong password, they are locked out for 15 minutes.

- **Evidence:** `authentication/api_views.py` — `_is_locked_out()` method

### 5. Tamper-proof audit log

Once an audit record is created, it cannot be edited or deleted. This is enforced by overridden `save()` and `delete()` methods.

- **Evidence:** `organizations/models.py` — lines 180-192

## 6. Session security

Session cookie expires when browser closes, cannot be read by JavaScript, and only sent over HTTPS in production.

- **Evidence:** `config/settings.py` — `SESSION_COOKIE_HTTPONLY = True`,  
`SESSION_EXPIRE_AT_BROWSER_CLOSE = True`

## 7. JWT token security

Short expiry (30 minutes), rotated refresh tokens, old tokens blacklisted.

- **Evidence:** `config/settings.py` — `SIMPLE_JWT` settings

## 8. Clickjacking protection

Our pages cannot be loaded inside an iframe on another website.

- **Evidence:** `X_FRAME_OPTIONS = 'DENY'`

## 9. Content sniffing protection

Browsers cannot guess content types — prevents certain types of attacks.

- **Evidence:** `SECURE_CONTENT_TYPE_NOSNIFF = True`

## 10. Cross-schema protection

User IDs are stored as IntegerField instead of ForeignKey because PostgreSQL cannot create foreign keys across schemas. This is a security-conscious design decision.

- **Evidence:** `performed_by_id = models.IntegerField()`

## 11. Pending access (approval workflow)

New users cannot just log in — an admin must approve them first.

- **Evidence:** `authentication/oidc_backend.py` — `create_user()` returns None

### How PendingAccess works in detail:

This solves a real problem: what happens when someone has a Keycloak account (can log in) but doesn't have a profile in Django (no role, no ministry)?

```
Step 1: User logs in through Keycloak successfully
Step 2: Keycloak redirects to Django with a valid token
Step 3: Django looks for the user in its OWN database
Step 4: If NOT found → PendingAccess record is created
Step 5: User sees: "Your account is awaiting approval"
Step 6: Super Admin sees the request and approves/rejects it
```

Step 7: If approved → Django creates the actual user profile  
Step 8: User can now log in successfully

Without PendingAccess, the user would just get "access denied" with no explanation.

#### Code evidence (authentication/oidc\_backend.py):

```
try:
 user = CustomUser.objects.get(username=keycloak_username)
 return user # Found them – let them in

except CustomUser.DoesNotExist:
 # Keycloak says they're real, but Django doesn't know them
 PendingAccess.objects.get_or_create(
 keycloak_id=keycloak_id,
 defaults={
 'username': keycloak_username,
 'email': keycloak_email,
 }
)
 return None # User sees "waiting for approval" message
```

## 12. LoginAttempt brute-force tracking

Every login attempt — success or failure — is permanently recorded. Django counts failed attempts from the same username in the last 15 minutes and blocks after 5 failures.

- **Evidence:** `authentication/models.py` — `LoginAttempt` model
- **Evidence:** `authentication/api_views.py` — `_is_locked_out()` helper method

## 14. Password validation

Users must have passwords at least 8 characters, cannot be common passwords, cannot be similar to username.

- **Evidence:** `AUTH_PASSWORD_VALIDATORS` in `settings.py`

## 15. Warning for default secret key

If production is running with a weak secret key, Django prints a warning.

- **Evidence:** Security checks in `settings.py`

## Things that might look like issues (but are actually correct)

The panel may notice these things and think they are security problems. Here is the honest explanation for each:

| Thing that might look like an issue                            | The real situation                                                                                      |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Django's password fields are empty/null for Keycloak users     | This is correct — passwords live in Keycloak, not Django. Django's password is set to an unusable value |
| PendingAccess records exist with no role assigned              | These are users Keycloak knows about but Django hasn't approved yet. Normal and expected                |
| LoginAttempt records keep growing                              | This is correct — they are a permanent security record and should never be deleted                      |
| Web "forgot password" might still point to Django's reset page | This is a real issue to fix — redirect it to Keycloak's forgot-password page instead                    |
| <code>/api/auth/login/</code> can be called without a token    | This is correct — you obviously cannot require a token BEFORE logging in                                |
| Some endpoints return 403 (Forbidden) instead of 404           | This is intentional — returning 404 would reveal whether the URL exists; 403 is more secure             |
| Multiple failed login attempts from the same IP                | Both Keycloak and Django track and block these after 5 failures                                         |

## PART 9: DATABASE DESIGN — How our tables are organized

Public schema (shared across ALL ministries):

```
tenants_ministry → List of all ministries (MOH, MOF, etc.)
tenants_domain → Domain to ministry mapping (moh.localhost → MOH)
authentication_customuser → All users from all ministries
django_session → Login sessions
...
```

Each ministry schema (one per ministry):

```
assets_asset → Assets belonging to this ministry
assets_assetcategory → Asset categories (ICT, Furniture, etc.)
organizations_orgunit → Org hierarchy (Ministry → Agency → Facility)
organizations_masterdata → Dropdown options (funding sources, etc.)
organizations_auditlog → Audit trail for this ministry
...
```

Why this design?

**Question:** "Why not just have one table with a `ministry_id` column?"

**Answer:** Two reasons:



1. **Security** — If a programmer forgets to filter by `ministry_id`, they accidentally see ALL ministries' data. With schemas, they can ONLY see their own ministry's data. The database enforces this, not the code.
  2. **Performance** — Each schema's tables are smaller. Querying 1,000 assets in MOH's table is faster than querying 100,000 assets in one big table and filtering by `ministry_id`.
- 

## PART 10: OTHER TECHNOLOGIES — What they are and where we use them

### Django REST Framework (DRF)

**What:** A library that makes building APIs easy. **Where:** Every `api_views.py` file uses DRF's `APIView`, `Response`, and `status` classes. **Why:** Writing JSON APIs by hand is tedious. DRF handles JSON conversion, error handling, authentication, and permissions automatically.

### django-cors-headers

**What:** Allows the Flutter mobile app (running on a phone) to call our Django API (running on a laptop).

**Where:** `MIDDLEWARE` includes `corsheaders.middleware.CorsMiddleware`. Settings include `CORS_ALLOW_ALL_ORIGINS`. **Why:** Without CORS, the mobile browser would block API calls because they come from different addresses.

### mozilla-django-oidc

**What:** Handles the Keycloak OIDC handshake (redirects, code exchange, token verification). **Where:** `urlpatterns` includes `path('oidc/', include('mozilla_django_oidc.urls'))`. Settings include `OIDC_OP_*_ENDPOINT` URLs. **Why:** Writing OIDC from scratch is extremely complex and error-prone. Mozilla (the Firefox company) maintains this library.

### drf-yasg

**What:** Generates Swagger API documentation automatically. **Where:** `/api/docs/` in the browser shows the documentation page. **Why:** Other groups (Group 2-10) need to understand our API. Instead of writing documentation manually, drf-yasg reads our code and creates an interactive page.

### django-rest-framework-simplejwt

**What:** Creates and verifies JWT tokens for the mobile app. **Where:** `SIMPLE_JWT` settings in `settings.py`. Used in `LoginAPIView`, `RefreshTokenAPIView`, etc. **Why:** JWT is the standard way for mobile apps to authenticate. SimpleJWT handles token creation, refresh, blacklisting, and verification.

### python-decouple

**What:** Reads secrets from `.env` file. **Where:** `SECRET_KEY = config('SECRET_KEY')` in `settings.py`. **Why:** You never hardcode passwords or secret keys in your code. If someone views your code on GitHub, they can't steal your secrets.

### requests

**What:** A library for making HTTP calls from Python. **Where:** Used to call Keycloak Admin API (creating users in Keycloak from Django). **Why:** Django needs to communicate with Keycloak server-to-server.

## pytest-django

**What:** A testing framework. **Where:** Currently not used extensively (test files are placeholders). **Why:** Allows writing automated tests that verify the system works correctly.

---

## PART 11: DEPLOYMENT TO CLOUD — What, where, how

What is "the cloud"?

The cloud = someone else's computer that runs 24/7.

Instead of running `python manage.py runserver` on your laptop (which you turn off at night), you rent a server from a company like:

- **AWS** (Amazon Web Services)
- **DigitalOcean** (\$5/month basic server)
- **Linode** or **Vultr**

What you need for production:

```
[User's Browser]
 ↓
[Domain Name: moh.mof.go.tz]
 ↓
[DNS – points domain to server IP]
 ↓
[NGINX or Apache – web server]
 (handles HTTPS certificates, static files, security)
 ↓
[Gunicorn or uWSGI – runs Django]
 ↓
[PostgreSQL Database]
```

Do you use 0.0.0.0 in production?

**NO.** In production:

- NGINX listens on `0.0.0.0:80` (HTTP) and `0.0.0.0:443` (HTTPS with SSL)
- Django runs behind NGINX, listening ONLY on `127.0.0.1:8000`
- NGINX forwards (proxies) requests from the internet to Django

The 0.0.0.0:8000 thing is ONLY for your development testing when you access Django from your phone.

What about domains?

In production:

```
https://moh.mof.go.tz → Routes to Ministry of Health's schema
https://mof.mof.go.tz → Routes to Ministry of Finance's schema
https://admin.mof.go.tz → Routes to public schema (Super Admin)
```

Each domain gets an SSL certificate (for HTTPS — the padlock icon).

## PART 12: JWT TOKENS — Access token, refresh token, how they work

What is a JWT token?

JWT stands for **JSON Web Token**. It is a string of text that proves who you are. Think of it like a **digital ID card** that the mobile app carries around.

The mobile app logs in once, gets this ID card, and shows it with every request so Django knows: "Oh, this is user X, they are a Ministry Admin for MOH."

What does a JWT look like?

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxfQ.s5xLm8vF9kZn6w
```

PART 1  
HEADER

PART 2  
PAYLOAD

PART 3  
SIGNATURE

Three parts separated by dots:

| Part        | Name             | What it contains                                                               | Can you read it?         |
|-------------|------------------|--------------------------------------------------------------------------------|--------------------------|
| First part  | <b>Header</b>    | What type of token and how it was signed (e.g., HS256)                         | Yes (base64 decoded)     |
| Second part | <b>Payload</b>   | The actual data: user_id, role, ministry_schema, expiration time               | Yes (base64 decoded)     |
| Third part  | <b>Signature</b> | A cryptographic seal that proves the token was issued by Django and not forged | No (requires secret key) |

**Important:** Parts 1 and 2 are just encoded (base64), NOT encrypted. Anyone can decode them and see the contents. But they CANNOT change them because the signature (part 3) would break.

How to SEE inside a JWT token:

Go to [jwt.io](https://jwt.io) and paste your token. It will show you:

```
// HEADER
{
 "alg": "HS256",
```

```
"typ": "JWT"
}

// PAYLOAD
{
 "user_id": 1,
 "role": "MINISTRY_ADMIN",
 "ministry_schema": "moh_schema",
 "exp": 1712345678, // ← Expiration time (Unix timestamp)
 "iat": 1712343878 // ← Issued at time
}
```

## Access token vs Refresh token

We have TWO tokens, not one:

### ACCESS TOKEN (short life)

Purpose: Proves who you are for API calls  
Valid for: 30 minutes  
Used in: Every API request header  
What if stolen? Only useful for 30 minutes

### REFRESH TOKEN (longer life)

Purpose: Gets you a NEW access token when the old one expires  
Valid for: 1 day  
Used in: POST /api/auth/refresh/  
What if stolen? Can be blacklisted (invalidated)

## The flow:

### Mobile app logs in

- Gets access token (30 min) + refresh token (1 day)
- Uses access token for 30 minutes of API calls
- Token expires (30 min passed)
- Mobile app calls: POST /api/auth/refresh/ with refresh token
- Django gives a NEW access token (another 30 min)
- Sometimes also gives a NEW refresh token (ROTATE\_REFRESH\_TOKENS = True)
- Old refresh token is blacklisted (cannot be used again)
- Cycle continues until 1 day passes
- After 1 day, user must log in again

## Code evidence — Where we configure token lifetimes:

```
config/settings.py — LINE 235-248
from datetime import timedelta

SIMPLE_JWT = {
```

```

 'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30), # ← 30 minutes
 'REFRESH_TOKEN_LIFETIME': timedelta(days=1), # ← 1 day
 'ROTATE_REFRESH_TOKENS': True, # ← Give new refresh token
each time
 'BLACKLIST_AFTER_ROTATION': True, # ← Old one stops working
 'ALGORITHM': 'HS256', # ← How we sign it
 'SIGNING_KEY': SECRET_KEY, # ← Uses Django's secret
key
 'AUTH_HEADER_TYPES': ('Bearer',), # ← Token prefix in header
}

```

How to change the token duration (if panel asks):

```

Change 30 minutes to 2 hours:
'ACCESS_TOKEN_LIFETIME': timedelta(hours=2)

Change 1 day to 7 days:
'REFRESH_TOKEN_LIFETIME': timedelta(days=7)

```

Why 30 minutes? Why not 1 year?

Short access tokens = more secure. If someone steals your token, they can only use it for 30 minutes. After that, it's useless. The refresh token lasts longer but can be blacklisted if stolen.

Code evidence — Where the token is created with extra data:

```

authentication/api_serializers.py – LINE 7-18
class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
 @classmethod
 def get_token(cls, user):
 token = super().get_token(user)
 # These extra fields are embedded INSIDE the token
 token['role'] = user.role
 token['ministry_schema'] = user.ministry_schema or ''
 token['full_name'] = user.get_full_name() or user.username
 return token

```

This means when the mobile app decodes the token, it can read the user's role and ministry without making an extra API call.

Code evidence — Mobile app login endpoint:

```

authentication/api_urls.py – LINE 18
path('auth/login/', LoginAPIView.as_view(), name='api_login'),
POST /api/auth/login/ → returns { access, refresh, user }

```

## Code evidence — How the login API creates the token

When the mobile app or other group calls `POST /api/auth/login/`, this code runs:

```
authentication/api_views.py
class LoginAPIView(APIView):
 permission_classes = [AllowAny] # No token needed to log in

 def post(self, request):
 username = request.data.get('username')
 password = request.data.get('password')

 user = authenticate(username=username, password=password)
 if user:
 # Create the token with extra info baked in
 refresh = RefreshToken.for_user(user)
 refresh['role'] = user.role # Role goes INTO the token
 refresh['ministry_schema'] = user.ministry_schema # Ministry too

 return Response({
 'access': str(refresh.access_token),
 'refresh': str(refresh),
 'user': {
 'username': user.username,
 'role': user.role,
 'ministry_schema': user.ministry_schema
 }
 })
 return Response({'error': 'Invalid credentials'}, status=401)
```

**What this means:** When a user logs in, Django:

1. Checks the username/password (`authenticate`)
2. Creates a JWT token with `RefreshToken.for_user(user)`
3. Adds `role` and `ministry_schema` as extra fields INSIDE the token
4. Returns the token AND the user info in one response

The Flutter app or other group never needs to make a second call to find out the user's role — it's already in the token.

---

## PART 13: INTEGRATION WITH OTHER GROUPS — How they use our API

### Our role

We are Group 1. Groups 2-10 depend on us. They need to call our API to:

- Verify users (is this token valid? what role do they have?)
- Get asset data for their systems
- Get organization data

## What do we share with them?

We share our **API endpoint URLs** and a **sample token**. We do NOT share our source code or database.

### Scenario 1: They just need to check if a user is real

Their system receives a token from a mobile user. They call our API to verify it.

#### What we give them:

```
API Endpoint: POST http://moh.platform.go.tz/api/auth/verify-token/
```

Headers:

```
Authorization: Bearer <the_token_their_user_provided>
```

```
Content-Type: application/json
```

Response (if valid):

```
{
 "valid": true,
 "role": "MINISTRY_ADMIN",
 "ministry_schema": "moh_schema",
 "full_name": "Amina Hassan",
 "email": "amina@moh.go.tz"
}
```

Response (if invalid/expired):

```
{
 "valid": false,
 "message": "Token is invalid or expired"
}
```

**They don't need our database. They don't need Django. They just need to make an HTTP request to our URL.**

### Scenario 2: They want asset data for reporting

#### What we give them:

```
API Endpoint: GET http://moh.platform.go.tz/api/assets/?status=ACTIVE
```

Headers:

```
Authorization: Bearer <valid_token>
```

Response:

```
{
 "count": 150,
 "results": [
 {"id": 1, "asset_number": "MOH-ICT-0001", "name": "Dell Laptop", ...},
 ...
]
}
```

```
]
}
```

Do we give them a token or do they get their own?

**They get their own token.** Here's how:

1. We create a user account for their system (like "group2\_service\_account")
2. We give them: username + password
3. Their system calls: POST /api/auth/login/ with those credentials
4. Our API returns: { access\_token, refresh\_token, user\_info }
5. They use the access\_token in all subsequent API calls
6. When token expires, they call /api/auth/refresh/ to get a new one

**The simple analogy:**

The **API** is the door. The **token** is the key to that door. You share the **door** (API URL) with other groups. They get their **own key** (token) by logging in themselves.

You do NOT hand them a token directly. You never say "here is your token." That would be like giving someone your house key — they'd have YOUR identity, not their own.

Here is exactly what you give them (one time, via WhatsApp or email):

1. Our server address: `http://172.16.20.232:8000`
2. Our API docs link: `http://172.16.20.232:8000/api/docs/`
3. A test account: `username: moh_admin`  
`password: Admin@123`

And here is what THEY do with it:

```
Step 1 – They call your login API themselves:
POST http://172.16.20.232:8000/api/auth/login/
{"username": "moh_admin", "password": "Admin@123"}

Step 2 – Your system gives THEM their own token:
{"access": "eyJ...their token...", "user": {"role": "MINISTRY_ADMIN"}}

Step 3 – They use that token on every request:
GET http://172.16.20.232:8000/api/assets/
Header: Authorization: Bearer eyJ...their token...
```

**So the flow is:** You give them the door address → they knock (login) → the system gives them their own key → they use that key on every future knock.

Do other groups go to the Keycloak login page?



**No.** This is important to understand.

Keycloak's login page is only for **web browser users** — real ministry staff using the website. The journey for web users is:

```
Web browser user → gets redirected to Keycloak login page → types password there
```

Other groups are **developers building systems**, not humans typing passwords in a browser. They use our **API login endpoint** directly (the `POST /api/auth/login/` shown above). That endpoint checks the username and password internally and gives back a JWT token. Keycloak's web page never appears.

Think of it this way:

- Keycloak's page = the front door of a government office (for real staff walking in)
- API login endpoint = a back door for systems talking to systems (no human needed)

Both doors lead to the same building — but other groups use the back door.

What if they want a public API (no token required)?

You already have this ability. Look at your views — some use `AllowAnonymous`, some use `IsAuthenticated`:

```
PUBLIC – no token needed (currently only the login endpoint):
class LoginAPIView(APIView):
 permission_classes = [AllowAnonymous]

PROTECTED – token required:
class AssetListCreateAPIView(APIView):
 permission_classes = [IsAuthenticated]
```

If another group asks you to make a specific endpoint public — for example, a read-only list of asset categories — you change one line:

```
class AssetCategoryListAPIView(APIView):
 permission_classes = [AllowAnonymous] # ← Change this line
```

**Should you make things public?** Only read-only, non-sensitive data. Never user data, never audit logs, never financial data. The rule is: if the data could embarrass the government or reveal security information, it must stay protected.

What if they use different technology?

**It doesn't matter.** Our API speaks HTTP + JSON. Every programming language can do this:

| Their technology | How they call our API                                                   |
|------------------|-------------------------------------------------------------------------|
| Python           | <code>requests.get(url, headers={'Authorization': 'Bearer ...'})</code> |
| PHP              | <code>curl</code> or <code>file_get_contents</code>                     |
| Java             | <code>HttpURLConnection</code> or <code>OkHttp</code>                   |
| JavaScript       | <code>fetch(url, {headers: {'Authorization': 'Bearer ...'}})</code>     |
| C# .NET          | <code>HttpClient</code>                                                 |
| Any language     | HTTP is universal — every language supports it                          |

### Sample Python code they would write:

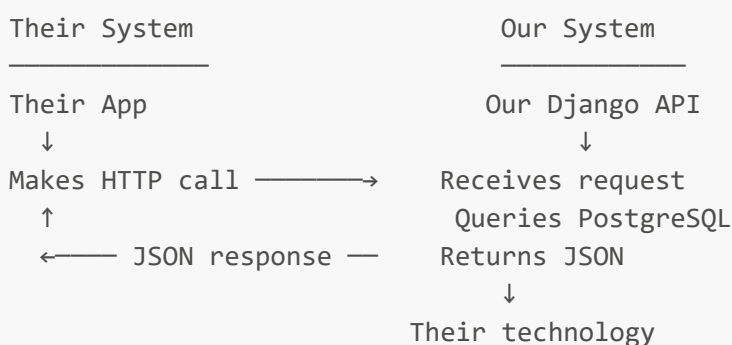
```
import requests

Step 1: Login to get a token
login_response = requests.post(
 'http://moh.platform.go.tz/api/auth/login/',
 json={'username': 'group2_user', 'password': 'given_password'}
)
token = login_response.json()['access']

Step 2: Use the token to get asset data
headers = {'Authorization': f'Bearer {token}'}
assets_response = requests.get(
 'http://moh.platform.go.tz/api/assets/',
 headers=headers
)
assets = assets_response.json()
print(f"Total assets: {assets['count']}")
```

### What if they don't use PostgreSQL?

**They don't need to.** Our database is inside our system. They never connect to our database. They only connect to our API (which is an HTTP endpoint). Our API queries our PostgreSQL database and returns JSON. They receive JSON. Their database can be anything — MySQL, MongoDB, SQL Server, or even Excel.



doesn't matter –  
they just read JSON

## What if they need a new feature or data?

They tell us: "We need to get all assets that expired this month." We add a new API endpoint or a filter parameter. We update the Swagger documentation at `/api/docs/`. They check the documentation and start using the new endpoint.

## What happens if we create a user but they don't add it?

They will call `/api/auth/verify-token/` with that user's token and get back: `{"valid": false, "message": "User not found"}`. They will know the user doesn't exist in our system.

## What happens if they add a user but we don't?

The user will authenticate at Keycloak successfully, but when Keycloak redirects to Django, Django will say: "This user is not registered in our system." A PendingAccess record will be created and the user will see: "Your account is not yet registered. Contact your administrator."

## Where is our API documentation?

`http://localhost:8000/api/docs/`

This is a Swagger UI page that shows EVERY endpoint with:

- The URL path (e.g., `/api/assets/`)
- What HTTP method to use (GET, POST, etc.)
- What data to send (JSON format)
- What you get back (JSON format)
- A "Try it out" button to test the API live

We tell other groups: "Go to this URL. All our APIs are documented there."

**How is this page automatically created?** In your `settings.py` and `config/urls.py`, you have `drf-yasg` installed, which reads your code and builds the documentation page:

```
config/settings.py – drf_yasg is in INSTALLED_APPS
INSTALLED_APPS = [
 ...
 'drf_yasg', # ← This is what creates the automatic documentation
]

config/urls.py – The Swagger page configuration
from drf_yasg.views import get_schema_view
from drf_yasg import openapi

schema_view = get_schema_view(
```

```
openapi.Info(
 title="GovAsset Platform API",
 default_version='v1',
 description="REST API for the Government Asset Management Platform",
),
public=True, # Anyone can view the docs (no login required)
)

urlpatterns += [
 path('api/docs/', schema_view.with_ui('swagger')),
]
```

Every time you add a new API endpoint, Swagger automatically adds it to the documentation page. You never write documentation manually.

**This means:** If another group asks "Can you add an API for X?" you:

1. Add the new endpoint to `api_urls.py` and `api_views.py`
2. Restart the server
3. The new endpoint appears on the Swagger page automatically
4. Tell them: "Check the docs again, it's there now"

What credentials do we give other groups for testing?

```
API Base URL: http://192.168.100.18:8000/api/
Username: test_group_user
Password: Test@123
```

They use these to log in and get a token. Simple.

Who gives the token to other groups? Django or Keycloak?

**Django gives the token.** Here is why:

```
Token type: JWT (JSON Web Token)
Issued by: Django (using SimpleJWT library)
Token contains: user_id, role, ministry_schema, expiration time
Signed with: Django's SECRET_KEY
```

Keycloak also issues tokens, but those Keycloak tokens are ONLY for web browser login. The tokens we give other groups for API access are Django tokens, created by SimpleJWT.

**Important:** Keycloak tokens and Django JWT tokens are DIFFERENT. They have different formats, different signatures, and different purposes:

**KEYCLOAK TOKEN (for web login)**

- Used between: Browser → Django → Keycloak
- Validates: "This user logged in through Keycloak"
- Format: OIDC id\_token (longer, more complex)

**DJANGO JWT TOKEN (for API access)**

- Used between: Mobile app → Django, Other groups → Django
- Validates: "This user has permission to call our API"
- Format: SimpleJWT access\_token (shorter, simpler)

What does an actual API endpoint look like? (Real example from our code)

When another group's developer asks "What URL do I call?" — here is exactly what we give them:

BASE URL (during development):

`http://192.168.100.18:8000/api/`

BASE URL (in production):

`https://moh.platform.go.tz/api/`

ENDPOINT: Get list of assets

`GET /api/assets/`

ENDPOINT: Get one specific asset

`GET /api/assets/{id}/`

Example: `GET /api/assets/5/`

ENDPOINT: Create a new asset

`POST /api/assets/`

ENDPOINT: Update an asset

`PUT /api/assets/{id}/`

ENDPOINT: Delete an asset

`DELETE /api/assets/{id}/`

**But we don't give them all endpoints individually!** We give them ONE URL:

**"Go to `http://192.168.100.18:8000/api/docs/` — all our APIs are listed there with a 'Try it out' button."**

The Swagger page (`/api/docs/`) automatically lists EVERY endpoint that exists. If we add a new endpoint tomorrow, it appears on the Swagger page automatically. The other group does NOT need us to send them a new document.

What attributes/fields do other groups need to send? (How do they know?)

Again, Swagger tells them. Here is what they see on the Swagger page for "Create Asset":

POST /api/assets/

Request body (JSON):

```
{
 "name": "Dell Latitude 5420", // Required – the asset name
 "asset_number": "MOH-ICT-0042", // Required – unique ID
 "category": 3, // Required – category ID from dropdown
 "status": "ACTIVE", // Required – one of: ACTIVE,
UNDER_MAINTENANCE, DECOMMISSIONED, DISPOSED
 "description": "Laptop for ICT officer", // Optional
 "serial_number": "SN123456789", // Optional
 "purchase_date": "2024-01-15", // Optional – format: YYYY-MM-DD
 "purchase_cost": "2500000.00", // Optional – decimal number as string
 "location": "MOH HQ, Room 305", // Optional
 "assigned_to": "amina@moh.go.tz", // Optional – user email
 "org_unit": 12 // Optional – org unit ID
}
```

The Swagger page also shows:

- Which fields are required vs optional
- What data type each field is (string, number, date)
- What the valid values are for dropdown fields
- Example values
- The exact JSON format

How do other groups access our API during development?

During development, we are all on the same college/local WiFi. Here is the setup:

OUR LAPTOP (Group 1):

IP: 192.168.100.18

Running: python manage.py runserver 0.0.0.0:8000

API available at: http://192.168.100.18:8000/api/

OTHER GROUP'S LAPTOP (Group 2-10):

Also on the same WiFi

They open their browser/postman/terminal

They call: GET http://192.168.100.18:8000/api/assets/

They get back: JSON data

**What if our IP changes?** We tell them: "My IP changed, here is the new one:

http://192.168.100.XXX:8000/api/"

**What if we are on different WiFi?** We cannot access each other. We would need to either:

1. Both connect to the same WiFi (college network)
2. Use a tool like ngrok (gives us a public URL that forwards to our laptop)

### 3. Deploy to a cloud server (production mode)

## Complete example: A group wants to integrate with us from scratch

Here is the ENTIRE process from start to finish:

STEP 1: We meet with the other group

Them: "We want to display our ministry's assets in our dashboard."

Us: "OK. Here is our API base URL and your test credentials."

STEP 2: We give them:

API Base URL: `http://192.168.100.18:8000/api/`

Username: `group2_service_account`

Password: `Test@123`

Swagger Docs: `http://192.168.100.18:8000/api/docs/`

STEP 3: They test the API manually (in their browser or Postman)

They open: `http://192.168.100.18:8000/api/docs/`

They click "Try it out" on `GET /api/assets/`

They see the JSON response with actual data

They understand the format

STEP 4: They write code in THEIR system

Let's say they use PHP (their system is built in PHP).

They write this code:

```
```php
<?php
// Step 1: Login to get token
$login = http_post('http://192.168.100.18:8000/api/auth/login/', [
    'username' => 'group2_service_account',
    'password' => 'Test@123'
]);
$token = json_decode($login)->access;

// Step 2: Get assets
$assets = http_get('http://192.168.100.18:8000/api/assets/', [
    'Authorization: Bearer ' . $token
]);
$data = json_decode($assets);

// Step 3: Display in their dashboard
foreach ($data->results as $asset) {
    echo $asset->name . " - " . $asset->status;
}
?>
```

STEP 5: They test. It works. They are integrated.

STEP 6 (later): We deploy to production. We tell them: "The new URL is `https://moh.platform.go.tz/api/`" They update one line in their code. Everything still works.

Do we also give them access to Keycloak?

****No.**** Other groups do NOT need Keycloak access. Here is why:

For API access, other groups use: Username + Password → Django → Django JWT token

They NEVER go through Keycloak for API access. They NEVER need to log into Keycloak admin. They NEVER need Keycloak credentials.

Keycloak is ONLY for web browser SSO login. API access uses Django's own JWT authentication.

What if our system is down? How does it affect other groups?

Their System → calls our API → no response (our server is down) → Their dashboard shows "Service unavailable" → Their users see a timeout error → They cannot get fresh data until our system is back up → They CAN use their last cached data (if they saved it)

This is why we need production deployment (24/7 server). During development, we only run when we are testing.

How does the mobile (Flutter) app use our API?

The Flutter app is ALSO an "other group" – it just happens to be built by our team, not another group. The Flutter app:

1. Has our API base URL hardcoded: `http://192.168.100.18:8000/api/`
2. Shows a login screen: username + password fields
3. Calls `POST /api/auth/login/` with those credentials
4. Gets back a JWT token (access + refresh)
5. Stores the token on the phone (secure storage)
6. Adds Authorization: Bearer to EVERY request
7. When token expires, silently refreshes it
8. If refresh fails, shows login screen again

The Flutter app NEVER opens a browser to Keycloak. It uses the direct API login (same as other groups would).

Complete list of ALL API endpoints other groups can use

These are defined in `authentication/api\_urls.py` and `assets/api\_urls.py`:

Method	URL	Purpose
POST	`/api/auth/login/`	Get JWT token (username + password)
POST	`/api/auth/refresh/`	Get new access token (using refresh token)
POST	`/api/auth/logout/`	Blacklist token (log out)
POST	`/api/auth/verify-token/`	Check if a token is valid
GET	`/api/auth/profile/`	Get current user's profile
GET/POST	`/api/assets/`	List all assets / Create new asset
GET/PUT/DELETE	`/api/assets/{id}/`	Get / Update / Delete one asset
GET	`/api/org-units/`	List organization units
GET	`/api/dashboard/stats/`	Dashboard statistics
GET	`/api/audit-log/`	View audit log entries

All these are documented at `/api/docs/` with example requests and responses.

Where is the API configuration in our code?

The API routing is configured in TWO files:

****File 1 – `authentication/api\_urls.py` (lists every API endpoint):****

```
```python
This file maps URLs to their handler views
urlpatterns = [
 # Authentication
 path('auth/login/', LoginAPIView.as_view()),
 path('auth/logout/', LogoutAPIView.as_view()),
 path('auth/refresh/', RefreshTokenAPIView.as_view()),
 path('auth/verify-token/', VerifyTokenAPIView.as_view()),
 path('auth/me/', MeAPIView.as_view()),

 # Assets
 path('assets/', AssetListCreateAPIView.as_view()),
 path('assets/<int:asset_id>', AssetDetailAPIView.as_view()),
 path('assets/categories/', AssetCategoryListAPIView.as_view()),

 # Organization
 path('org-units/', OrgUnitListAPIView.as_view()),

 # Audit logs
 path('audit-logs/', AuditLogListAPIView.as_view()),

 # Dashboard statistics
 path('dashboard/stats/', DashboardStatsAPIView.as_view()),
]
```

**File 2 — `config/urls.py` (connects the API to the main project):**

```
urlpatterns = [
 # ... all the web page URLs ...

 # This ONE line attaches ALL API endpoints:
 path('api/', include('authentication.api_urls')),

 # Swagger documentation
 path('api/docs/', schema_view.with_ui('swagger')),
]
```

The line `path('api/', include(...))` puts `api/` in front of every URL in `api_urls.py`. So `auth/login/` becomes `/api/auth/login/`. That is why all your APIs start with `/api/`.

**How other groups see this:** They never look at these files. They just go to `/api/docs/` and see everything listed automatically.

## PART 14: Why does each ministry need its own domain?

**Question:** "Why can't we use one domain like `platform.go.tz/ministry/1/`?"

**Answer:** Because django-tenants works by reading the DOMAIN from the URL, not the PATH.

How django-tenants identifies the tenant:

```
URL: https://moh.platform.go.tz/assets/
 ↑
 django-tenants reads this part
 Looks up "moh.platform.go.tz" in Domain table
 Finds: this belongs to Ministry of Health
 Switches database to moh_schema
```

Why not use URL path instead? (e.g., `/ministry/1/`)

```
This would NOT work with django-tenants:
URL: https://platform.go.tz/ministry/1/assets/
 ↑
 django-tenants doesn't look here
 It ONLY looks at the domain name
```

django-tenants is designed specifically for the "each tenant has its own domain" model. This is actually the standard for SaaS products:

- `company1.salesforce.com`
- `company2.slack.com`
- `moh.github.com`

But we also handle IP access:

For mobile API access, users hit our server by IP address (not domain). We set:

```
SHOW_PUBLIC_IF_NO_TENANT_FOUND = True
```

When Django can't find a matching domain (because the user connected via IP), it falls back to the public schema. This is why the mobile app works even without domains.

---

## PART 15: PRESENTATION TIPS — How to present well

Before the panel:

1. **Run the project** — Make sure `python manage.py runserver 0.0.0.0:8000` works. Show it running.
2. **Have test data** — Create a test ministry, test user, test assets so you can click through the UI.
3. **Open Keycloak admin** — Show Keycloak's admin page (<http://localhost:8180>) with users listed there.
4. **Open PostgreSQL** — Show the database schemas: `\dn` in psql shows `moh_schema`, `mof_schema`, etc.

Common panel questions and how to answer:

### Q: "What problem does your system solve?"

"Currently, government ministries track assets on paper or Excel. There is no central system. When an asset moves between ministries, there is no record. Our system gives each ministry its own secure space to track assets, with a shared super admin who can see everything."

### Q: "Why PostgreSQL and not MySQL?"

"Because only PostgreSQL supports schemas. We need schemas to give each ministry its own private set of tables. MySQL doesn't have this feature."

### Q: "How is data kept separate between ministries?"

"Each ministry has its own PostgreSQL schema — basically its own set of tables. When a user from Ministry of Health logs in, the system only connects to MOH's schema. Even if there's a bug in the code, the database won't show another ministry's data."

### Q: "What if one ministry has millions of assets?"

"Each schema is independent. MOH having 2 million assets doesn't slow down MOF's queries. Each ministry's tables only contain their own data."

### Q: "How does the mobile app authenticate?"

"The mobile app calls our API with username and password. Django returns a JWT token that expires in 30 minutes. The mobile app sends this token with every request. When it expires, they use a refresh token to get a new one."

**Q: "What happens if Keycloak is down?"**

"Users who are already logged in can continue. New users cannot log in. The mobile app's existing tokens continue to work for 30 minutes. After that, they cannot refresh."

**Q: "What is the most important security feature?"**

"Database-level isolation. Each ministry's data lives in a separate PostgreSQL schema. Even if someone finds a bug in our code, the database itself prevents cross-ministry access."

**Q: "How do you know who changed an asset?"**

"Every action — create, update, delete, login, logout — is recorded in the AuditLog. The record cannot be modified or deleted. It captures who did it, what they changed, the old values, and the new values."

**Q: "How do you add a new ministry?"**

"The Super Admin fills a form with the ministry name, schema name, and domain. When they click submit, Django creates a new PostgreSQL schema, runs all migrations inside it, creates the root organization unit, and sets up the domain mapping. The whole process takes about 2 seconds."

If you don't know an answer:

"That's a good question. I have not tested that scenario yet. Based on how the system is designed, I believe it would work like this: [your best guess]. But I would need to verify that before giving a confident answer."

This is honest and shows understanding without lying.

For the demo:

1. Start with the **login page** — show the SSO button
2. Log in as **Super Admin** — show ministry list
3. Show **onboarding a new ministry** — explain schemas
4. Show **asset list** — demonstrate search, filter, pagination
5. Show **audit log** — demonstrate tamper-proof records
6. Show **Swagger API docs** (/api/docs/) — demonstrate the API
7. Show **PostgreSQL** — run `\dn` to show schemas

---

## PART 16: QUICK REFERENCE — Key files and what they do

File	What it does	Why it exists
<code>config/settings.py</code>	All configuration — apps, database, security, auth	The brain of the project
<code>config/urls.py</code>	Maps URLs to view functions	The roadmap — matches URLs to code
<code>tenants/models.py</code>	Ministry and Domain models	Defines what a tenant (ministry) is

File	What it does	Why it exists
<code>tenants/views.py</code>	Web pages for managing ministries	Super Admin manages ministries here
<code>authentication/models.py</code>	CustomUser, LoginAttempt, PendingAccess	User accounts beyond Django's defaults
<code>authentication/views.py</code>	Login/logout pages	The web login page
<code>authentication/api_views.py</code>	API login, refresh, verify token	Mobile app authentication
<code>authentication/api_urls.py</code>	API URL routes	Maps API URLs to API views
<code>authentication/api_serializers.py</code>	Converts data to JSON for API	Translates database to API format
<code>authentication/api_permissions.py</code>	Who can access which API endpoint	Security for APIs
<code>authentication/api_exception_handler.py</code>	Consistent error format for API	Makes errors predictable
<code>authentication/decorators.py</code>	Role checking for web views	Security for web pages
<code>authentication/middleware.py</code>	Sets schema on every request	Ensures correct data isolation
<code>authentication/oidc_backend.py</code>	Bridges Keycloak to Django users	The Keycloak connection
<code>authentication/pagination.py</code>	Splits lists into pages	Handles "next page" logic
<code>assets/models.py</code>	Asset and AssetCategory models	The core data — what is an asset?
<code>assets/api_views.py</code>	Asset CRUD API endpoints	Mobile app manages assets here
<code>assets/views.py</code>	Asset web pages	Browser manages assets here
<code>organizations/models.py</code>	OrgUnit, MasterData, AuditLog	Org hierarchy, dropdowns, audit trail
<code>organizations/views.py</code>	Org hierarchy web pages	Browser views org tree
<code>organizations/api_views.py</code>	Org tree, audit log, dashboard API	Mobile app reads org data

File	What it does	Why it exists
<code>organizations/master_data_views.py</code>	Dropdown management pages	Admin configures dropdown options
<code>templates/shared/base.html</code>	Main layout with sidebar	Every page uses this template

## PART 17: COMPLETE REQUEST-TO-RESPONSE FLOW

Web browser flow (full example):

1. User types: `http://moh.localhost:8000/assets/`  

↑
2. Operating system asks DNS: "What is moh.localhost?"
  - /etc/hosts file says: 127.0.0.1 moh.localhost
  - So the request goes to your own computer (localhost)
3. Django's runserver receives the request on port 8000
4. Django goes through MIDDLEWARE in order:
  - a) TenantMainMiddleware:
    - "The domain is moh.localhost. Let me find the tenant."
    - Queries `tenancy_domain` table: "Who owns moh.localhost?"
    - Finds: Ministry of Health (`moh_schema`)
    - Sets connection to `moh_schema`
  - b) SecurityMiddleware:
    - Adds security headers to the response later
  - c) SessionMiddleware:
    - Loads the session for this browser
  - d) CorsMiddleware:
    - Checks if this is a cross-origin request
  - e) CommonMiddleware:
    - Handles URL redirects (e.g., `/assets` → `/assets/`)
  - f) AuthenticationMiddleware:
    - "Is this user logged in?"
    - Loads the user from the session
  - g) Our SchemaMiddleware:
    - "What is this user's ministry\_schema?"
    - Sets `request.schema_name = 'moh_schema'`
  - h) SessionRefresh (mozilla-django-oidc):
    - Checks if Keycloak session is still valid

5. URL dispatch:  
Django matches /assets/ against urlpatterns:  
→ Finds: path('assets/', asset\_list\_view, name='asset\_list')
6. Decorators check (before view runs):  
@login\_required\_custom: "Is user logged in? Yes → continue"  
@role\_required('MINISTRY\_ADMIN'): "What is user's role? Yes → continue"
7. View function runs:  
with schema\_context('moh\_schema'):  
    assets = Asset.objects.all() → Only MOH's assets  
Returns HTML page with list of assets
8. Response goes back through middleware (reverse order)
9. Browser receives HTML and renders it

### Mobile API flow (full example):

1. Flutter app sends: GET http://192.168.100.18:8000/api/assets/  
Header: Authorization: Bearer eyJhbGciOiJI...
2. Django's middleware processes the request (same as above)
3. URL dispatch matches: /api/assets/  
→ Finds: AssetListCreateAPIView
4. Permission classes check:  
IsAuthenticated: "Is the JWT token valid? Yes → continue"  
HasMinistrySchema: "Does user have a ministry? Yes → continue"  
CanManageAssets: "GET is read-only, all roles can read → continue"
5. View runs:  
with schema\_context('moh\_schema'):  
    assets = Asset.objects.all()  
    serializer = AssetSerializer(assets, many=True)  
    return Response(serializer.data) → JSON response
6. Flutter receives JSON and shows it in the mobile app

## PART 18: GLOSSARY — Simple definitions

Term	Simple meaning
<b>Schema</b>	A private folder inside the PostgreSQL database. Each ministry has one.
<b>Tenant</b>	One customer in a multi-tenant system. For us, one ministry = one tenant.

Term	Simple meaning
<b>OIDC</b>	OpenID Connect — a standard way for one website to let another website handle login. Google, Facebook, and Microsoft use this.
<b>JWT</b>	JSON Web Token — a string of text that proves who you are. Like a digital ID card with an expiration date.
<b>SSO</b>	Single Sign-On — log in once, access many systems. Keycloak is our SSO provider.
<b>DRF</b>	Django REST Framework — makes building APIs easy.
<b>CORS</b>	Cross-Origin Resource Sharing — tells browsers it's OK to let the Flutter app call our API.
<b>Middleware</b>	Code that runs on every request before your view. Like airport security — everyone goes through it.
<b>Decorator</b>	A wrapper around a function that checks something first (like "is the user logged in?").
<b>Serializer</b>	Converts database data to JSON (for API) and JSON back to database data.
<b>Mixin</b>	A pre-built class you add to your model to give it extra features automatically. TenantMixin gives us schema_name for free.
<b>REST API</b>	A way for programs to talk to each other over the internet. Our Flutter app uses the REST API to get and update data.
<b>Claims</b>	Information inside a JWT token about the user (username, role, email, etc.).
<b>Realm</b>	In Keycloak: a collection of users, clients, and configurations. Ours is called "govasset".
<b>Search path</b>	PostgreSQL setting that tells it which schemas to look in when running queries. django-tenants sets this automatically.

## PART 19: REMEMBER THIS FOR THE PANEL

The panel wants to know:

1. **You understand your own project** — Not just how to use it, but WHY you made each choice
2. **You can explain technical concepts simply** — If your supervisor asks "what is multi-tenancy?", you can explain it without reading from a script
3. **You know the weaknesses** — Be honest about what's not perfect (no tests, no rate limiting)
4. **You can demo it live** — Practice clicking through the UI without freezing

Key sentences to memorize:

"Each ministry gets its own PostgreSQL schema — a private set of database tables. This means Ministry of Health data is physically separate from Ministry of Finance data, enforced by the database, not just by our code."

"Keycloak handles the password checking. Our Django app never sees the user's password. Keycloak tells us: 'This user is who they say they are.' Then we check our database: 'Does this user exist in our system?'"



"JWT tokens expire after 30 minutes with a 1-day refresh token. This limits damage if a token is stolen — the thief can only use it for 30 minutes."

"Our audit log is append-only. Once a record is created, it cannot be changed or deleted. This is required for government compliance and asset tracking accountability."

The one-paragraph answer — If the panel asks "How does your security work end to end?"

Memorize this paragraph. It covers authentication, tokens, multi-tenancy, and audit in one confident answer:

*"Our system has two types of users: web browser users and API users. Web users log in through Keycloak, which handles password checking and brute-force lockout. API users, including our mobile app and other groups' systems, log in through our own API endpoint, which has its own five-attempt lockout using our LoginAttempt model. Both types of login produce a JWT token containing the user's identity, role, and ministry. Every subsequent request attaches that token, and our permission classes check it before any data is touched. The multi-tenancy middleware switches the database to the correct ministry's private schema before any query runs. Every action is permanently recorded in an audit log that cannot be modified or deleted by anyone, including the Super Admin. This is all configured in settings.py, oidc\_backend.py, api\_views.py, and the models in tenants and organizations."*

This answer covers Keycloak, JWT, brute-force, multi-tenancy, and audit — everything they are likely to ask about.

## PART 20: CSS QUICK REFERENCE — What to change when the panel asks

The panel may ask: "Change this text to red" or "Make the background blue". Here is how to do it fast.

Quickest way: Inline CSS (highest priority)

Inline CSS beats any file. Add `style="..."` to any HTML tag:

```
<!-- Change text color -->
<h1 style="color:red;">This text will be red</h1>
<p style="color:blue;">Blue text</p>
Green text

<!-- Change background color -->
<div style="background-color:yellow;">Yellow background</div>
<td style="background:#fff3cd;">Light yellow cell</td>

<!-- Change text size -->
<p style="font-size:18px;">Bigger text</p>
<p style="font-size:14px;">Smaller text</p>
<h1 style="font-size:32px;">Custom heading size</h1>

<!-- Change font weight (boldness) -->
<p style="font-weight:bold;">Bold text</p>
<p style="font-weight:normal;">Normal weight</p>

<!-- Change padding (space inside) -->
```

```

<div style="padding:20px;">More space inside</div>
<div style="padding:10px 15px;">10 top/bottom, 15 left/right</div>

<!-- Change margin (space outside) -->
<div style="margin-bottom:20px;">Push things below down</div>

<!-- Change border -->
<div style="border:2px solid red;">Red border</div>
<div style="border:1px dashed #ccc;">Dashed grey border</div>

<!-- Change width -->
<div style="width:50%;">Half width</div>
<div style="max-width:800px;">Max 800px wide</div>

<!-- Multiple styles together -->
<div style="color:red; background:yellow; font-size:18px; padding:15px; border:1px
solid orange;">
 All at once
</div>

```

## Our project's CSS variable names (used everywhere)

We defined these in `static/css/style.css`. Use them in inline styles:

```

<!-- Text colors -->
style="color:var(--accent);" /* Blue accent color */
style="color:var(--danger);" /* Red for errors */
style="color:var(--success);" /* Green for success */
style="color:var(--warning);" /* Orange/amber for warnings */
style="color:var(--text-muted);" /* Grey for secondary text */
style="color:var(--text-primary);" /* Default text color */

<!-- Background colors -->
style="background:var(--bg-page);" /* Page background */
style="background:var(--success-bg);" /* Light green bg */
style="background:var(--danger-bg);" /* Light red bg */
style="background:var(--warning-bg);" /* Light yellow bg */
style="background:var(--accent);color:#fff;" /* Blue bg + white text */

<!-- Border colors -->
style="border-color:var(--danger-border);"
style="border-color:var(--success-border);"

<!-- Border radius (rounded corners) -->
style="border-radius:var(--radius-sm);" /* Small rounding */
style="border-radius:var(--radius-md);" /* Medium rounding */
style="border-radius:var(--radius-lg);" /* Large rounding */
style="border-radius:var(--radius-xl);" /* Extra large */

<!-- Shadows -->
style="box-shadow:var(--shadow-sm);" /* Small shadow */

```

```
style="box-shadow:var(--shadow-md);" /* Medium shadow */
style="box-shadow:var(--shadow-lg);" /* Large shadow */
```

## Common color names vs our project colors

Panel says...	Use this inline CSS
"Make this red"	<code>style="color:red;"</code> or <code>style="color:var(--danger);"</code>
"Make background yellow"	<code>style="background:yellow;"</code> or <code>style="background:var(--warning-bg);"</code>
"Make this green"	<code>style="color:green;"</code> or <code>style="color:var(--success);"</code>
"Make this blue"	<code>style="color:blue;"</code> or <code>style="color:var(--accent);"</code>
"Make text bigger"	<code>style="font-size:20px;"</code>
"Make text smaller"	<code>style="font-size:12px;"</code>
"Make this bold"	<code>style="font-weight:bold;"</code>
"Add spacing"	<code>style="padding:15px;"</code> or <code>style="margin:10px 0;"</code>
"Add border"	<code>style="border:1px solid #ccc;"</code>

## Custom CSS classes we have (defined in style.css)

Do NOT use Bootstrap classes like `text-danger` or `bg-primary` — Bootstrap CSS is NOT loaded. Instead, use our custom classes:

Class	What it does
<code>.btn</code>	Makes a link/button look like a styled button
<code>.btn-primary</code>	Blue button
<code>.btn-outline</code>	Button with border (no fill)
<code>.btn-danger-outline</code>	Red outlined button
<code>.card</code>	White box with shadow and rounded corners
<code>.badge</code>	Small colored label
<code>.badge-active</code>	Green badge (asset is active)
<code>.badge-maint</code>	Orange badge (under maintenance)
<code>.badge-decomm</code>	Grey badge (decommissioned)
<code>.data-table</code>	Full-width table with borders
<code>.alert-danger</code>	Red message box
<code>.alert-success</code>	Green message box

Class	What it does
<code>.alert-warning</code>	Yellow message box
<code>.alert-info</code>	Blue message box
<code>.form-control</code>	Input field styling
<code>.form-select</code>	Dropdown styling
<code>.page-header</code>	Title area at top of page
<code>.filter-bar</code>	Search/filter row
<code>.empty-state</code>	"No data" message box
<code>.stat-card</code>	Dashboard statistic box
<code>.sidebar</code>	Left navigation panel

## Bootstrap Icons (yes, we have these)

We load Bootstrap Icons (not Bootstrap CSS). Use them like this:

```

<i class="bi bi-plus-circle"></i> <!-- Plus icon -->
<i class="bi bi-pencil"></i> <!-- Edit/pencil icon -->
<i class="bi bi-trash"></i> <!-- Delete/trash icon -->
<i class="bi bi-search"></i> <!-- Search icon -->
<i class="bi bi-download"></i> <!-- Download icon -->
<i class="bi bi-arrow-left"></i> <!-- Left arrow -->
<i class="bi bi-check-circle"></i> <!-- Check mark -->
<i class="bi bi-exclamation-triangle"></i> <!-- Warning triangle -->
<i class="bi bi-gear"></i> <!-- Settings gear -->
<i class="bi bi-person"></i> <!-- User icon -->
<i class="bi bi-box"></i> <!-- Box/asset icon -->
<i class="bi bi-building"></i> <!-- Building/ministry icon -->
<i class="bi bi-shield-lock"></i> <!-- Security/lock icon -->

```

## Where files are located

What you want to change	File to edit
Global styles (all pages)	<code>static/css/style.css</code>
A single page's layout	The <code>.html</code> template file in <code>templates/</code>
A single element	Use inline <code>style="..."</code> on that HTML tag

If they ask you to find what CSS class controls something:

Open the page in a browser → Right-click the element → Inspect (or F12) → Look at the "Styles" tab → It shows you which CSS file and line number controls it.

If they ask you to add a new page:

1. Create `.html` file in the correct `templates/` folder
2. Use existing classes: `class="page-header"`, `class="card"`, `class="btn btn-primary"`
3. Use inline styles for unique styling
4. All templates extend `base.html` which loads `style.css` automatically

---

## PART 21: DATABASE DEEP DIVE — Tables, schemas, shared vs private

Why do we never start PostgreSQL?

PostgreSQL runs as a **Windows service** — it starts automatically when Windows starts. You don't need to run it manually.

```
Check it: Open "Services" (Windows search → type "Services")
→ Look for "postgresql-x64-16" (or similar name)
→ Status should be "Running"
→ Startup type: "Automatic" (starts when Windows boots)
```

```
If it's not running: Right-click → Start
```

When you run `python manage.py runserver`, Django automatically connects to PostgreSQL using the settings in `config/settings.py`. You never need to type `pg_ctl start` or anything like that.

How many tables do we have? And their purpose?

We have **10 main tables** across our system:

#	Table name	Schema	Purpose
1	<code>tenants_ministry</code>	Public	List of all ministries (MOH, MOF, MOE...)
2	<code>tenants_domain</code>	Public	Maps domains to ministries (moh.localhost → MOH)
3	<code>authentication_customuser</code>	Public	All users from ALL ministries
4	<code>authentication_loginattempt</code>	Public	Failed login records (brute force tracking)
5	<code>authentication_pendingaccess</code>	Public	Users waiting for admin approval
6	<code>django_session</code>	Public	Browser session storage
7	<code>assets_asset</code>	Per-ministry	All assets for that ministry
8	<code>assets_assetcategory</code>	Per-ministry	Asset categories (ICT, Furniture, Vehicles...)
9	<code>organizations_orgunit</code>	Per-ministry	Organization hierarchy (Ministry → Agency → Facility)

#	Table name	Schema	Purpose
10	<code>organizations_auditlog</code>	Per-ministry	Every action recorded (cannot be deleted)
+	<code>organizations_masterdata</code>	Per-ministry	Dropdown options (funding sources, etc.)

**Wait — tables 1-6 are in the PUBLIC schema (one copy for everyone) Tables 7-10+ are in EACH MINISTRY'S PRIVATE schema (separate copy per ministry)**

If we have 5 ministries, we have: 6 public tables + (5 × 4 private tables) = 26 tables total.

What columns does each table have?

### 1. tenants\_ministry (who are our ministries?)

```
id → Auto number (1, 2, 3...)
name → "Ministry of Health"
schema_name → "moh_schema" (used by PostgreSQL)
created_at → When this ministry was added
```

### 2. tenants\_domain (how URLs map to ministries)

```
id → Auto number
domain → "moh.localhost" (the URL people type)
tenant_id → Points to which Ministry this domain belongs to
is_primary → Is this the main domain for this ministry?
```

### 3. authentication\_customuser (all users in the system)

```
id → Auto number
username → Login name (e.g., "moh_admin")
email → Email address
password → Hashed password (never the real password!)
role → SUPER_ADMIN, MINISTRY_ADMIN, AGENCY_MANAGER, FACILITY_CLERK, AUDITOR
ministry_schema → "moh_schema" (which ministry they belong to)
keycloak_id → Their ID in Keycloak (links the two systems)
is_active → True/False (can they log in?)
date_joined → When they registered
```

### 4. assets\_asset (the main data — what we track)

```
id → Auto number
name → "Dell Latitude 5420"
```

```

asset_number → "MOH-ICT-0042" (unique barcode number)
category → Which category (points to assetcategory table)
status → ACTIVE, UNDER_MAINTENANCE, DECOMMISSIONED, DISPOSED
description → "Laptop for ICT officer"
serial_number → Manufacturer's serial number
purchase_date → When it was bought
purchase_cost → How much it cost
location → Where it is physically
assigned_to → Who is using it (email)
org_unit → Which organization unit it belongs to
created_at → When this record was added
updated_at → When it was last changed

```

## Why do some tables live in the public schema and others in private schemas?

This is the most important database design decision. Here is the simple rule:

```

PUBLIC SCHEMA = Things that are shared across ALL ministries
- Users (a user belongs to one ministry but the user table is shared)
- Ministries (the list of all ministries)
- Domains (the URL-to-ministry mapping)
- Login attempts (tracking failed logins globally)
- Pending access (approval queue for new users)
- Sessions (browser login sessions)

PRIVATE SCHEMA = Things that belong to ONE ministry only
- Assets (MOH's laptops are not MOF's laptops)
- Categories (each ministry has their own categories)
- Org units (each ministry has their own structure)
- Audit logs (each ministry's actions are private)

```

## The key reason: Data isolation is enforced by the database.

```

WRONG APPROACH (one big table with ministry_id):
assets_asset table:
 id name ministry_id
 1 Dell Laptop 1 (MOH)
 2 Office Chair 2 (MOF)
 3 Hospital Bed 1 (MOH)

Query: Asset.objects.all()
→ Returns ALL 3 rows from all ministries
→ BUG! A programmer forgot to filter by ministry_id
→ MOH sees MOF's data. Security breach.

OUR APPROACH (separate schemas):
MOH schema → assets_asset table:
 id name
 1 Dell Laptop

```

```
3 Hospital Bed
```

```
MOF schema → assets_asset table:
```

```
id name
2 Office Chair
```

```
Query: Asset.objects.all() (while in MOH schema)
→ Returns only 2 rows (MOH's data)
→ Cannot see MOF's data even if we forget to filter
→ The DATABASE ITSELF prevents cross-ministry access
```

**Analogy:** Think of the public schema as the building's LOBBY — everyone shares it (entrance, directory, security desk). Each ministry's private schema is their OWN OFFICE — other ministries cannot walk in.

Why is the users table in the public schema?

Because a user's identity crosses ministries:

```
Super Admin → manages ALL ministries → needs to see everything
Ministry Admin → manages ONE ministry → sees only their schema
But ALL users are listed in one table so we can:
1. Check "does this username already exist?"
2. Super Admin can see all users across all ministries
3. Prevent duplicate accounts
```

If users were in private schemas, two ministries could accidentally create the user "admin" — and we wouldn't know.

But wait — if users are in the public schema, can MOH users see MOF users?

**No.** The `CustomUser` model has a `ministry_schema` field. When we query users, we filter by schema:

```
In code, when listing users for a ministry admin:
users = CustomUser.objects.filter(ministry_schema='moh_schema')
This only returns MOH users, even though the table is shared
```

This is called **application-level filtering** (the code filters). It is less secure than schema-level isolation, which is why we only use it for the users table (which needs to be shared).

---

## PART 22: MAKING THE SYSTEM LIVE — From development to production

What needs to happen to make our system live?

Right now, our system only runs on our laptop. To make it live (accessible 24/7 to real users), we need:



#### WHAT WE HAVE NOW (development):

- Our laptop → `python manage.py runserver 0.0.0.0:8000`
- Only works when laptop is on
- Only works on our WiFi
- Only we can access it
- No proper URL (just IP address)
- No HTTPS (no padlock icon)
- Laptop might overheat if running 24/7

#### WHAT WE NEED (production):

- A cloud server (AWS / DigitalOcean / Vultr)
- Running: Ubuntu Linux + PostgreSQL + Django
- Accessible at: `https://moh.platform.go.tz`
- Running: 24/7, never stops
- Secure: HTTPS padlock, firewall, automatic backups
- Fast: Good internet connection, proper server

## What happens if we don't use proper domains?

If we only use `127.0.0.1:8000` or `192.168.x.x:8000`:

#### Problem 1: django-tenants cannot identify the ministry

- django-tenants reads the domain from the URL
- If URL is `127.0.0.1:8000` → no domain found
- Falls back to public schema
- All users see the public schema (not their ministry's data)
- The whole multi-tenancy system breaks!

#### Problem 2: No real SSL/HTTPS

- Modern mobile apps REQUIRE HTTPS for API calls
- Without HTTPS, the Flutter app will refuse to connect
- Browsers show "Not Secure" warning

#### Problem 3: No proper URL

- Users cannot remember "`192.168.100.18:8000`"
- They expect "`moh.go.tz`" or "`govasset.go.tz`"
- Looks unprofessional for a government system

## What code changes are needed for production?

### 1. ALLOWED\_HOSTS in settings.py:

```
Currently (development):
ALLOWED_HOSTS = ['192.168.100.18', '172.16.20.232', '10.187.165.150',
 'localhost', '127.0.0.1', '.localhost']

Change to (production):
ALLOWED_HOSTS = ['.platform.go.tz', # All subdomains
 'platform.go.tz', # Main domain
 '192.168.100.18'] # Keep IP for testing
```

## 2. Add real domain entries in the Domain table:

```
Currently: Domain.objects.create(domain='moh.localhost', tenant=moh)
Change to: Domain.objects.create(domain='moh.platform.go.tz', tenant=moh)
```

## 3. Set DEBUG = False:

```
Currently:
DEBUG = True

Change to (production):
DEBUG = False
```

## 4. Configure HTTPS/SSL:

```
Add to settings.py for production:
SECURE_SSL_REDIRECT = True # Force HTTPS
SESSION_COOKIE_SECURE = True # Only send session cookies over HTTPS
CSRF_COOKIE_SECURE = True # Only send CSRF cookies over HTTPS
```

## 5. Set up a proper web server (NGINX or Apache):

```
NGINX configuration (simplified):

server {
 listen 443 ssl;
 server_name moh.platform.go.tz;

 ssl_certificate /etc/ssl/certs/platform.go.tz.crt;
 ssl_certificate_key /etc/ssl/private/platform.go.tz.key;

 location / {
 proxy_pass http://127.0.0.1:8000;
 proxy_set_header Host $host;
 }
}
```

```
location /static/ {
 alias /var/www/govasset/static/;
}
}
```

How will other groups access our system in production?

```
Other Group → Their Browser → https://moh.platform.go.tz/
 ↓
 DNS server
 (maps domain to server IP)
 ↓
 Cloud Server (AWS/DigitalOcean)
 ↓
 NGINX (handles HTTPS)
 ↓
 Gunicorn (runs Django)
 ↓
 PostgreSQL Database
```

**They type:** `https://moh.platform.go.tz/` in their browser **They see:** Our application, with proper URL, proper SSL, professional look

**For API access:** Same thing, but use:

```
https://moh.platform.go.tz/api/assets/
Instead of:
http://192.168.100.18:8000/api/assets/
```

What about our own `etc/hosts` file?

Currently, we use the `etc/hosts` file to make `moh.localhost` work:

```
127.0.0.1 moh.localhost
127.0.0.1 mof.localhost
127.0.0.1 admin.localhost
```

In production, we don't need this because real DNS servers handle the mapping:

```
moh.platform.go.tz → DNS → 104.28.45.12 (the server's public IP)
```

The `etc/hosts` file is ONLY for development (since we don't own real domains yet).

## How to see what domain a user is using (code evidence):

```
tenants/views.py – The tenant detection logic
from django_tenants.utils import get_tenant_model

def get_tenant_from_request(request):
 domain = request.get_host() # Gets: "moh.localhost" or "192.168.100.18:8000"
 domain_model = get_tenant_model()
 try:
 tenant = domain_model.objects.get(domains__domain=domain.split(':')[0])
 return tenant
 except domain_model.DoesNotExist:
 return None # No matching domain → public schema
```

## What does django-tenants do when it finds a domain?

```
django-tenants internally does this:
1. Reads: request.get_host() → "moh.localhost"
2. Queries: Domain.objects.get(domain="moh.localhost")
3. Gets: tenant = moh_ministry (schema_name = "moh_schema")
4. Runs: connection.set_schema("moh_schema")
5. Now ALL queries use: SET search_path TO moh_schema, public;
```

---

## PART 23: DEVELOPMENT SETUP — How web + mobile work together

### How did the web developer and mobile developer access the same data?

This is a common panel question. Here is how we worked together:

SCENARIO: Two developers on different laptops, different roles

#### Web Developer (Group 1A):

Laptop A  
IP: 192.168.100.18  
Runs: Django + PostgreSQL  
Port: 8000

His job:  
Build web pages  
Build APIs  
Manage database

#### Mobile Developer (Group 1B):

Laptop B  
IP: 192.168.100.45  
Runs: Flutter + Emulator  
No database

His job:  
Build mobile screens  
Call existing APIs  
Display data on phone

**The mobile developer does NOT run Django or PostgreSQL.** The mobile developer only runs Flutter (mobile app framework). To test, the mobile developer:

```
Step 1: Connect to the same WiFi (both laptops on college network)
Step 2: Web developer runs: python manage.py runserver 0.0.0.0:8000
Step 3: Web developer tells mobile: "My IP is 192.168.100.18"
Step 4: Mobile developer puts this URL in the Flutter app code:
```

```
const API_BASE_URL = 'http://192.168.100.18:8000/api/';
```

```
Step 5: Mobile developer runs the Flutter app on his phone emulator
Step 6: The emulator calls: GET http://192.168.100.18:8000/api/assets/
Step 7: Web developer's Django receives the request and returns JSON
Step 8: Mobile developer's Flutter app displays the data
```

### What if the web developer's IP changes?

```
Web developer runs: ipconfig
→ Sees new IP: 192.168.100.55
→ Tells mobile developer: "My IP changed to 192.168.55"
→ Mobile developer updates one line in Flutter code
→ Everything works again
```

### What if they are on different WiFi?

```
→ They cannot connect directly
→ Solutions:
 a) Both join the same WiFi (college network, cafe, etc.)
 b) Web developer uses ngrok (creates a public URL):
 ngrok http 8000
 → Gives: https://abc123.ngrok.io
 → Mobile uses that URL instead
 c) Deploy Django to a temporary cloud server
```

Does the mobile developer need to install Django?

**No.** The mobile developer only needs:

```
Flutter SDK (to build the mobile app)
VS Code or Android Studio (to write code)
A phone or emulator (to test)
An internet connection (to reach the web developer's laptop)
```

The mobile developer NEVER needs:

Python or Django  
PostgreSQL  
Keycloak  
Our project's source code (Django part)

The mobile developer only needs the API URL and the documentation.

How does the mobile developer know what API endpoints exist?

1. Web developer runs Django with Swagger enabled
2. Mobile developer opens in browser: <http://192.168.100.18:8000/api/docs/>
3. Mobile developer sees ALL endpoints with descriptions
4. Mobile developer clicks "Try it out" to test each endpoint
5. Mobile developer sees exactly what JSON is returned
6. Mobile developer writes Flutter code that parses that JSON

**This is why Swagger is important!** Without Swagger, the mobile developer would need to ask: "What URL do I call for assets? What fields are returned? What format is the date?" With Swagger, they find everything themselves.

What if the mobile developer makes changes that need new API features?

```
Mobile developer: "I need an API that returns only assets that expired this month."
Web developer: "OK, I'll add it."
 → Web developer adds a filter: /api/assets/?expired_this_month=true
 → Swagger updates automatically
 → Mobile developer refreshes Swagger page
 → Mobile developer sees the new filter
 → Mobile developer starts using it
```

Why can't the mobile developer run everything locally?

The mobile developer COULD run Django locally, but:

1. Setting up PostgreSQL + Keycloak is complex
2. They would need all the demo data
3. Their laptop might be slow (Flutter + Django + PostgreSQL)
4. Any data they create would be on their laptop, not shared
5. It's easier to use one central API that everyone shares

**This is the same model used in real companies:**

Frontend team (mobile/web) → calls APIs from → Backend team (server)  
Backend team maintains the server and database  
Frontend teams just consume the APIs

What does the Flutter code look like for calling our API?

```
// Flutter code (simplified) – This is what the mobile developer writes

import 'package:http/http.dart' as http;
import 'dart:convert';

class ApiService {
 final String baseUrl = 'http://192.168.100.18:8000/api/';
 String? token;

 // Step 1: Login
 Future<void> login(String username, String password) async {
 final response = await http.post(
 Uri.parse('${baseUrl}auth/login/'),
 body: {'username': username, 'password': password},
);
 final data = json.decode(response.body);
 token = data['access']; // Save the JWT token
 }

 // Step 2: Get assets
 Future<List<Asset>> getAssets() async {
 final response = await http.get(
 Uri.parse('${baseUrl}assets/'),
 headers: {'Authorization': 'Bearer $token'},
);
 final data = json.decode(response.body);
 return data['results'].map((json) => Asset.fromJson(json)).toList();
 }
}
```

The Flutter developer's job is to:

1. Call the API URL
2. Receive the JSON response
3. Parse it and display it beautifully on the phone screen
4. Handle errors (no internet, token expired, etc.)

The Flutter developer does NOT need to know:

- What database we use
- How multi-tenancy works
- What Keycloak is

- How Python/Django works

They just need the API URL and the JSON format.

How do OTHER groups (Group 2-10) work with us?

Same model as the mobile developer:

Other Group's Developer:

- Gets our API URL + test credentials
- Opens Swagger: `http://192.168.100.18:8000/api/docs/`
- Tests endpoints
- Writes code in their language (PHP, Java, C#, etc.)
- Calls our API during development (same WiFi)
- When we deploy to production, switches to production URL

**The API is our contract with the world.** As long as the API keeps working the same way, other groups don't care what technology we use behind it.

## PART 24: COMPLETE FILE-BY-FILE EXPLANATION — Every folder and every file

This part explains the ENTIRE project, folder by folder, file by file. After reading this, if someone mentions any file, you will know exactly what it does.

### Project structure overview

```
govasset_platform/
├── manage.py # Entry point – everything starts here
├── config/ # Central settings, URLs, server entry
│ ├── settings.py # ALL configuration in one file
│ ├── urls.py # URL routing map
│ ├── asgi.py # For async deployment
│ └── wsgi.py # For production deployment
├── authentication/ # Users, login, API, security
│ ├── models.py # CustomUser, PendingAccess, LoginAttempt
│ ├── views.py # Login/logout web pages
│ ├── user_views.py # User CRUD web pages
│ ├── dashboard_views.py # Dashboard stats
│ ├── pending_access_views.py # Approve/reject new users
│ ├── api_views.py # Mobile app login/refresh/verify API
│ ├── api_serializers.py # Convert data to/from JSON
│ ├── api_permissions.py # Who can access each API
│ ├── api_exception_handler.py # Standard error format
│ ├── api_urls.py # API endpoint routing
│ ├── decorators.py # Security guards for web pages
│ ├── middleware.py # Sets schema on every request
│ ├── oidc_backend.py # Keycloak ↔ Django bridge
│ └── keycloak_admin.py # Talk to Keycloak admin API
```



```

├── pagination.py # Split lists into pages
├── assets/ # Asset register app
│ ├── models.py # Asset and AssetCategory models
│ └── views.py # Asset web pages (list, create, detail,
edit, delete)
├── api_views.py # Asset API for mobile app
├── organizations/ # Org hierarchy, master data, audit
│ ├── models.py # OrgUnit, MasterData, AuditLog
│ ├── views.py # Org unit web pages
│ ├── master_data_views.py # Dropdown options management
│ └── api_views.py # Org + dashboard API
├── tenants/ # Multi-tenancy management
│ ├── models.py # Ministry and Domain models
│ └── views.py # Ministry onboarding web pages
├── templates/ # All HTML pages (26 files)
│ ├── shared/ # Base layout (sidebar, topbar)
│ ├── authentication/ # Login, user management pages
│ ├── assets/ # Asset pages (list, form, detail)
│ ├── tenants/ # Ministry management pages
│ ├── dashboard/ # Dashboard page
│ └── organizations/ # Org unit, master data pages
├── static/ # CSS files
│ └── css/
│ └── style.css # All custom styles (758 lines)

```

## FOLDER 1: `config/` — The project brain

This folder contains EVERYTHING that makes the project work: settings, URLs, server entry points. Without this folder, Django wouldn't know what to do.

### `config/__init__.py`

Just tells Python "this folder is a package." Empty file.

### `config/settings.py` — The MOST important file (390 lines)

**Purpose:** ONE file that controls EVERYTHING — which apps are installed, which database to use, how authentication works, what middleware runs, how logging works, security settings.

**Think of it as:** The control panel of an airplane. Every switch, every setting, every configuration lives here.

### Key sections explained simply:

```

Lines 28-70: Which apps are shared vs per-ministry
SHARED_APPS = [
 'django_tenants', # The multi-tenancy library itself
 'tenants', # Ministry model (public schema)
 'authentication', # Users, auth (public schema – one user table for all)
 'rest_framework', # DRF – makes APIs easy
 'corsheaders', # Lets Flutter call our API from a different address

```

```

 'mozilla_django_oidc', # Handles Keycloak handshake
 'drf_yasg', # Creates Swagger docs automatically
]

TENANT_APPS = [
 'organizations', # Org hierarchy, audit logs (per-ministry)
 'assets', # Asset register (per-ministry)
]

```

```

Lines 135-143: Database connection
DATABASES = {
 'default': {
 'ENGINE': 'django_tenants.postgresql_backend',
 # ↑ This is the magic! django-tenants adds SET search_path before every
query
 }
}

```

```

Lines 217-235: JWT token settings
SIMPLE_JWT = {
 'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30), # Token dies in 30 min
 'REFRESH_TOKEN_LIFETIME': timedelta(days=1), # Refresh lasts 1 day
 'ROTATE_REFRESH_TOKENS': True, # New refresh each time
 'BLACKLIST_AFTER_ROTATION': True, # Old refresh stops
working
}

```

```

Lines 461-510: Logging
LOGGING = {
 # Writes to TWO files:
 # django.log – normal messages
 # security.log – IMPORTANT: security events (logins, failures, access denied)
}

```

### config/urls.py — The roadmap (169 lines)

**Purpose:** Maps every URL to the code that handles it. When someone types a URL, Django checks this file to find which function to run.

**Think of it as:** A restaurant menu — each dish (URL) has a kitchen (function) that prepares it.

```

Example – how URL routing works:
urlpatterns = [
 path('login/', login_view, name='login'),

```

```
When user types: http://moh.localhost:8000/login/
Django calls: login_view() function

path('assets/', asset_list_view, name='asset_list'),
When user types: http://moh.localhost:8000/assets/
Django calls: asset_list_view() function

path('api/', include('authentication.api_urls')),
All API URLs start with /api/... and are defined in api_urls.py

]
```

### Main URL groups:

URL pattern	What it does	Who uses it
/login/, /logout/	Web login/logout pages	Browser users
/dashboard/	Main dashboard	Logged-in users
/assets/...	Asset CRUD pages	Ministry admins, clerks
/users/...	User management	Admins
/ministries/...	Ministry onboarding	Super Admin
/api/...	All REST API endpoints	Mobile app, other groups
/api/docs/	Swagger documentation	Everyone
/oidc/...	Keycloak SSO handshake	Browser users

### config/asgi.py and config/wsgi.py

**Purpose:** Entry points for running Django on a server. ASGI is for modern async servers, WSGI is for traditional servers.

**When you use them:** NEVER in development. Only when deploying to production (cloud server). The cloud server (like Gunicorn) reads one of these files to start Django.

**Simplified:** When you type `python manage.py runserver`, Django ignores these files. When you type `gunicorn config.wsgi:application`, it uses wsgi.py.

## FOLDER 2: authentication/ — The most important folder

This folder handles EVERYTHING related to users, login, security, and APIs. It is the biggest folder (20+ files). If you understand this folder, you understand 70% of the project.

### authentication/\_\_init\_\_.py

Empty package marker.

### authentication/apps.py

**Purpose:** Tells Django this folder is an app named "authentication".

## authentication/models.py — User database design (199 lines)

**Purpose:** Defines THREE database tables: CustomUser, PendingAccess, LoginAttempt.

### 1. CustomUser (the main user table):

```
class CustomUser(AbstractUser):
 # Django's default user has: username, password, email, first_name, last_name
 # We ADDED these extra fields:
 role = models.CharField(
 choices=[# A user can only be ONE of these:
 'SUPER_ADMIN', # Top-level – manages all ministries
 'MINISTRY_ADMIN', # Manages ONE ministry
 'AGENCY_MANAGER', # Manages one agency within a ministry
 'FACILITY_CLERK', # Manages assets at one facility
 'AUDITOR', # Can ONLY read, cannot create/edit/delete
]
)
 ministry_schema = models.CharField() # "moh_schema" – ties user to their
 ministry
 keycloak_id = models.CharField() # Links to Keycloak account
 phone = models.CharField() # Phone number
```

### 2. PendingAccess (users waiting for approval):

```
class PendingAccess(models.Model):
 # Created when someone tries to log in via Keycloak
 # but doesn't have a Django account yet
 email = models.EmailField()
 keycloak_id = models.CharField()
 status = models.CharField() # PENDING → APPROVED or REJECTED
 # Admin must approve before they can log in
```

### 3. LoginAttempt (brute-force protection):

```
class LoginAttempt(models.Model):
 MAX_ATTEMPTS = 5 # After 5 failures...
 LOCKOUT_MINUTES = 15 # ...locked for 15 minutes

 username = models.CharField()
 ip_address = models.CharField()
 attempts = models.IntegerField(default=0)
 locked_until = models.DateTimeField(null=True)

 def is_locked(self):
```

```
Returns: "Is this user currently locked out?"
return self.locked_until and self.locked_until > now()
```

### authentication/views.py — Login/logout pages (145 lines)

**Purpose:** Shows the login page and handles logging out.

**login\_view:** Shows a page with a "Sign in with Government SSO" button. When clicked, it redirects to Keycloak. Also displays "Your account is pending approval" messages.

**logout\_view:** When user clicks logout:

1. Writes audit log: "User X logged out"
2. Destroys Django session
3. Redirects to Keycloak logout page (so Keycloak also forgets them)

### Key code — how we detect failed logins:

```
def _is_locked_out(username, ip_address):
 attempt = LoginAttempt.objects.filter(
 username=username, ip_address=ip_address
).first()
 if attempt and attempt.is_locked:
 return attempt.minutes_remaining # "Locked for 12 more minutes"
 return None
```

### authentication/user\_views.py — User management (403 lines)

**Purpose:** Web pages for creating, editing, activating/deactivating, and resetting passwords for users.

### Views and what they do:

View	What it does	Who can access
<code>user_list_view</code>	Shows all users, filtered by ministry	Admins
<code>user_create_view</code>	Creates user in Keycloak + Django simultaneously	Admins
<code>user_edit_view</code>	Edits user details, syncs to Keycloak	Admins
<code>user_toggle_active_view</code>	Enables/disables a user account	Admins
<code>user_reset_password_view</code>	Resets password in Django + Keycloak	Admins

### Key code — creating a user in BOTH systems:

```
def user_create_view(request):
 if request.method == 'POST':
 # Step 1: Create in Keycloak first
 keycloak = KeycloakAdminService()
```

```

kc_id = keycloak.create_user(form_data) # Returns Keycloak user ID

Step 2: Create in Django
user = CustomUser.objects.create(
 username=form_data['username'],
 keycloak_id=kc_id, # ← Links Django user to Keycloak user
 role=form_data['role'],
 ministry_schema=form_data['ministry_schema'],
)
If Keycloak fails → Django does NOT create the user (rollback)

```

### authentication/dashboard\_views.py — Dashboard stats (145 lines)

**Purpose:** Shows the main dashboard with statistics and expiry warnings.

#### dashboard\_view:

- Super Admin sees: "Total ministries: 5, Total users: 120, Total assets: 5,000"
- Ministry users see: "Your assets: 500, Expired: 12, Expiring soon: 25"

```

def _get_ministry_stats(ministry_schema):
 with schema_context(ministry_schema):
 # This ONLY queries their ministry's schema!
 total_assets = Asset.objects.count()
 expired = Asset.objects.filter(asset_expiry_date__lt=date.today())
 expiring_30 = Asset.objects.filter(asset_expiry_date__range=[today,
30_days])
 return {'total': total_assets, 'expired': expired.count()}

```

### authentication/pending\_access\_views.py — Access approvals (261 lines)

**Purpose:** Super Admin reviews users who tried to log in but weren't registered yet.

View	What it does
<code>pending_access_list_view</code>	Shows list of all pending/approved/rejected requests
<code>pending_access_review_view</code>	Approve or reject a specific request
<code>pending_access_clear_view</code>	Delete old rejected requests (30+ days)

### authentication/api\_views.py — Mobile app authentication (473 lines)

**Purpose:** REST API endpoints that the Flutter mobile app and other groups call. This is the MOST important file for API authentication.

#### Classes (each handles one API endpoint):

Class	URL	What it does
LoginAPIView	POST /api/auth/login/	Takes username + password → returns JWT tokens
RefreshTokenAPIView	POST /api/auth/refresh/	Takes old refresh token → returns new tokens
MeAPIView	GET /api/auth/me/	Returns current user's profile
VerifyTokenAPIView	GET/POST /api/auth/verify-token/	Checks if a token is valid
LogoutAPIView	POST /api/auth/logout/	Blacklists refresh token

### Key code — Login with brute force protection:

```
class LoginAPIView(TokenObtainPairView):
 def post(self, request):
 username = request.data.get('username')
 ip = get_client_ip(request)

 # Check 1: Is this user locked out?
 locked = _is_locked_out(username, ip)
 if locked:
 return Response({'error': f'Locked {locked} minutes'}, status=429)

 # Check 2: Is the password correct?
 serializer = self.get_serializer(data=request.data)
 if serializer.is_valid():
 _clear_failed_attempts(username, ip) # Success → reset counter
 return Response(serializer.validated_data)
 else:
 _record_failed_attempt(username, ip) # Failure → increment counter
 return Response({'error': 'Invalid credentials'}, status=401)
```

### authentication/api\_serializers.py — Data formatters (222 lines)

**Purpose:** Converts between Python objects and JSON format. The Flutter app sends/recieves JSON — serializers handle the conversion.

### Key classes:

Serializer	What it converts
CustomTokenObtainPairSerializer	Adds role, ministry_schema INSIDE the JWT token
UserProfileSerializer	User data → JSON for /api/auth/me/
AssetSerializer	Asset data → JSON for /api/assets/
AuditLogSerializer	Audit log → JSON

## Key code — Embedding extra data in JWT:

```
class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
 @classmethod
 def get_token(cls, user):
 token = super().get_token(user)
 # These fields are embedded INSIDE the token
 # So the mobile app can read them WITHOUT making an extra API call
 token['role'] = user.role
 token['ministry_schema'] = user.ministry_schema
 token['full_name'] = user.get_full_name()
 return token
```

## authentication/api\_permissions.py — API bouncers (117 lines)

**Purpose:** Checks "Does this user have permission to call this API?" before the API code runs.

### Classes:

Class	Who it lets through
IsSuperAdmin	Only SUPER_ADMIN role
IsMinistryAdmin	SUPER_ADMIN + MINISTRY_ADMIN
CanManageAssets	Everyone can READ (GET). Only admins can WRITE (POST/PUT/DELETE)
CanDeleteAssets	Only SUPER_ADMIN + MINISTRY_ADMIN can delete
CanViewAuditLogs	SUPER_ADMIN, MINISTRY_ADMIN, AUDITOR
HasMinistrySchema	Blocks users without a ministry (Super Admin is exempt)

```
class CanManageAssets(BasePermission):
 def has_permission(self, request, view):
 if request.method in ['GET', 'HEAD', 'OPTIONS']:
 return True # Anyone can READ
 # Only certain roles can WRITE
 return request.user.role in ['SUPER_ADMIN', 'MINISTRY_ADMIN',
 'AGENCY_MANAGER']
```

## authentication/api\_exception\_handler.py — Error formatter (36 lines)

**Purpose:** When an API error happens, DRF returns different error formats for different errors. This file makes ALL errors the same format so the Flutter app can handle them consistently.

```
def custom_exception_handler(exc, context):
 response = exception_handler(exc, context)
 if response is not None:
```



```

 response.data = {
 'error': True,
 'message': str(response.data.get('detail', 'An error occurred')),
 'code': getattr(exc, 'default_code', 'error'),
 'status': response.status_code,
 }
 return response

```

**Without this, errors look different each time:**

```

Error 1: {"detail": "Authentication credentials were not provided."}
Error 2: {"username": ["This field is required."]}
Error 3: {"non_field_errors": ["Unable to log in"]}

With this, ALL errors look the same:
{"error": true, "message": "...", "code": "...", "status": 401}

```

### authentication/api\_urls.py — API routing (89 lines)

**Purpose:** Maps all API URLs to their view classes.

```

urlpatterns = [
 path('auth/login/', LoginAPIView.as_view(), name='api_login'),
 path('auth/refresh/', RefreshTokenAPIView.as_view(), name='api_refresh'),
 path('auth/me/', MeAPIView.as_view(), name='api_me'),
 path('auth/verify-token/', VerifyTokenAPIView.as_view(), name='api_verify'),
 path('auth/logout/', LogoutAPIView.as_view(), name='api_logout'),
 path('assets/', AssetListCreateAPIView.as_view(), name='api_asset_list'),
 path('assets/<int:id>/', AssetDetailAPIView.as_view(),
name='api_asset_detail'),
 path('assets/categories/', AssetCategoryListAPIView.as_view()),
 path('org-units/', OrgUnitListAPIView.as_view()),
 path('audit-logs/', AuditLogListAPIView.as_view()),
 path('dashboard/stats/', DashboardStatsAPIView.as_view()),
]

```

**Result:** Flutter calls `POST /api/auth/login/` → Django runs `LoginAPIView`

### authentication/decorators.py — Web page bouncers (73 lines)

**Purpose:** Checks permissions BEFORE a web page loads. These are for browser pages (not APIs).

```

def login_required_custom(view_func):
 # "Guard" for web pages
 @wraps(view_func)
 def wrapper(request, *args, **kwargs):
 if not request.user.is_authenticated:

```

```

 messages.warning(request, "Please log in first.")
 return redirect('login')
 return view_func(request, *args, **kwargs)
return wrapper

def role_required(*allowed_roles):
 # "Guard" that checks user's role
 def decorator(view_func):
 @wraps(view_func)
 def wrapper(request, *args, **kwargs):
 if request.user.role not in allowed_roles:
 messages.error(request, "You don't have permission.")
 return redirect('dashboard')
 return view_func(request, *args, **kwargs)
 return wrapper
 return decorator

```

### Usage:

```

@login_required_custom # First guard: "Are you logged in?"
@role_required('SUPER_ADMIN') # Second guard: "Are you a Super Admin?"
def ministry_list_view(request):
 # This code ONLY runs if both guards say OK
 ...

```

### authentication/middleware.py — Schema setter (51 lines)

**Purpose:** On EVERY request, this sets `request.schema_name` so that all views know which ministry's data to use.

```

class SchemaMiddleware(MiddlewareMixin):
 def process_request(self, request):
 if request.user.is_authenticated:
 request.schema_name = request.user.ministry_schema
 request.user_role = request.user.role

```

**Why this is needed:** Without this, every view would need to look up the user's ministry schema from the database. This makes it available automatically.

### authentication/oidc\_backend.py — Keycloak bridge (146 lines)

**Purpose:** The MIDDLEMAN between Keycloak and Django. When Keycloak says "this user is real", this file checks if the user exists in Django.

```

class GovAssetOIDCBackend(OIDCAuthenticationBackend):
 def filter_users_by_claims(self, claims):

```

```

Keycloak says: "This user is authenticated"
Django says: "Let me find this user in MY database"
keycloak_id = claims.get('sub')
try:
 return CustomUser.objects.get(keycloak_id=keycloak_id)
except CustomUser.DoesNotExist:
 return self.UserModel.objects.none()

def create_user(self, request, user_claims):
 # Keycloak says: "This user is real but not in Django"
 # We do NOT auto-create. We create a PendingAccess request.
 PendingAccess.objects.create(
 email=user_claims.get('email'),
 keycloak_id=user_claims.get('sub'),
)
 return None # ← "User NOT allowed yet. Admin must approve."

```

### authentication/keycloak\_admin.py — Keycloak API (295 lines)

**Purpose:** Talks directly to Keycloak's admin API to create/update/delete users. When an admin creates a user in Django, this file also creates them in Keycloak.

```

class KeycloakAdminService:
 def _get_admin_token(self):
 # Logs into Keycloak as admin to get permission to create users
 response = requests.post(
 'http://localhost:8180/realms/master/protocol/openid-connect/token',
 data={
 'client_id': 'admin-cli',
 'username': settings.KEYCLOAK_ADMIN_USERNAME,
 'password': settings.KEYCLOAK_ADMIN_PASSWORD,
 'grant_type': 'password',
 }
)
 return response.json()['access_token']

 def create_user(self, data):
 # Creates user in Keycloak with role + ministry attributes
 response = requests.post(
 f'{KEYCLOAK_URL}/admin/realms/govasset/users',
 headers={'Authorization': f'Bearer {self.token}'},
 json={
 'username': data['username'],
 'email': data['email'],
 'enabled': True,
 'attributes': {
 'role': [data['role']], # ← Synced to Keycloak
 'ministry_schema': [data['ministry_schema']], # ← Synced!
 }
 }
)

```

```
 }
)
```

### authentication/pagination.py — Page splitter (36 lines)

**Purpose:** When you have 1000 assets, you don't show all 1000 on one page. This splits them into pages of 20.

```
def paginate_queryset(queryset, request, per_page=20):
 paginator = Paginator(queryset, per_page)
 page_number = request.GET.get('page', 1)
 page = paginator.get_page(page_number)
 return page, paginator
```

## FOLDER 3: assets/ — The asset register

This folder handles the MAIN DATA of the system — the assets themselves (laptops, vehicles, hospital beds, etc.).

### assets/models.py — Asset database design (245 lines)

**Purpose:** Defines what an "asset" is and what "categories" exist.

**AssetCategory:** Categories like ICT, Furniture, Vehicles, Medical Equipment.

```
class AssetCategory(models.Model):
 name = models.CharField() # "ICT Equipment"
 code = models.CharField() # "ICT" – used in asset numbers: MOH-ICT-0001
 is_active = models.BooleanField(default=True)
```

**Asset (30+ fields):** The main model with everything about an asset.

```
IDENTIFICATION
asset_number → "MOH-ICT-2025-0042" (auto-generated, unique barcode)
name → "Dell Latitude 5420"
serial_number → "SN123456789" (manufacturer's serial)

CATEGORY AND STATUS
category → Points to AssetCategory (e.g., ICT)
status → ACTIVE, UNDER_MAINTENANCE, DECOMMISSIONED, DISPOSED
condition → EXCELLENT, GOOD, FAIR, POOR, CRITICAL

LOCATION
org_unit → Which department/section
location_description → "3rd Floor, Room 305"
```

**DATES**

acquisition\_date → When bought  
 warranty\_expiry\_date → When warranty ends  
 asset\_expiry\_date → When it should be replaced  
 useful\_life\_years → Expected lifespan

**FINANCIAL**

acquisition\_cost → How much it cost  
 current\_value → Depreciated value  
 funding\_source → Where the money came from

**TRACKING**

registered\_by → Who created this record  
 created\_at → When created  
 updated\_at → When last modified

**Key computed properties (automatically calculated):**

```

@property
def is_expired(self):
 """Has this asset passed its expiry date?"""
 return self.asset_expiry_date and self.asset_expiry_date < date.today()

@property
def expires_soon(self):
 """Will this asset expire within 90 days?"""
 if self.asset_expiry_date:
 return 0 < (self.asset_expiry_date - date.today()).days <= 90
 return False

```

**assets/views.py — Asset web pages (577 lines)**

**Purpose:** Web pages for browser-based asset management.

View	URL	What it does
asset_list_view	/assets/	Shows paginated list with search + filters
asset_create_view	/assets/create/	Form to add new asset, auto-generates number
asset_detail_view	/assets/<id>/	Full asset detail with expiry warnings
asset_edit_view	/assets/<id>/edit/	Edit form, captures old/new values for audit
asset_delete_view	/assets/<id>/delete/	Delete (MINISTRY_ADMIN only)

**Key code — auto-generating asset numbers:**

```

def generate_asset_number(category_code, schema_name):
 # Result example: "MOH-ICT-2025-0042"

```

```
prefix = schema_name[:3].upper() # "MOH"
year = str(date.today().year) # "2025"
last_asset = Asset.objects.filter(
 asset_number__startswith=f'{prefix}-{category_code}-{year}'
).order_by('-asset_number').first()
next_num = int(last_asset.asset_number.split('-')[-1]) + 1 if last_asset else
1
return f'{prefix}-{category_code}-{year}-{next_num:04d}'
```

assets/api\_views.py — Asset API (628 lines)

**Purpose:** API endpoints that the Flutter mobile app uses to manage assets.

Class	URL	What it does
AssetListCreateAPIView	GET/POST /api/assets/	List assets or create new one
AssetDetailAPIView	GET/PUT/DELETE /api/assets/<id>/	Get/update/delete one asset
AssetCategoryListAPIView	GET /api/assets/categories/	Get list of categories for dropdowns

**Key code — creating an asset via API with automatic audit logging:**

```
class AssetListCreateAPIView(APIView):
 permission_classes = [IsAuthenticated, HasMinistrySchema, CanManageAssets]

 def post(self, request):
 serializer = AssetSerializer(data=request.data)
 if serializer.is_valid():
 with schema_context(request.user.ministry_schema):
 asset = serializer.save(
 registered_by_id=request.user.id,
 asset_number=generate_asset_number(...)
)
 # Auto-record audit log
 AuditLog.objects.create(
 action='CREATE',
 model_name='Asset',
 object_id=asset.id,
 new_value=serializer.data,
)
 return Response(serializer.data, status=201)
```

FOLDER 4: organizations/ — Org hierarchy, master data, audit

organizations/models.py — Three important models (230 lines)

## 1. OrgUnit (Organization Units):

```
class OrgUnit(models.Model):
 # Three-level hierarchy
 UNIT_TYPES = [
 ('MINISTRY', 'Ministry'), # Top level (e.g., Ministry of Health)
 ('AGENCY', 'Agency'), # Middle level (e.g., Tanzania Medicines
Authority)
 ('FACILITY', 'Facility'), # Bottom level (e.g., Muhimbili Hospital)
]
 name = models.CharField() # "Muhimbili National Hospital"
 code = models.CharField() # "MNH"
 unit_type = models.CharField(choices=UNIT_TYPES)
 parent = models.ForeignKey('self', null=True) # Points to parent unit
```

## 2. MasterData (dropdown options):

```
class MasterData(models.Model):
 # Configurable reference data per ministry
 CATEGORIES = [
 ('FUNDING_SOURCE', 'Funding Source'), # "Government Budget", "Donor
Funded"
 ('ACQUISITION_METHOD', 'Acquisition Method'), # "Purchase", "Donation"
 ('DISPOSAL_METHOD', 'Disposal Method'), # "Auction", "Transfer",
"Destroyed"
 ('COST_CENTRE', 'Cost Centre'), # "ICT Department", "HR
Division"
]
 category = models.CharField(choices=CATEGORIES)
 value = models.CharField() # "GOV_BUDGET"
 label = models.CharField() # "Government Budget"
 is_active = models.BooleanField(default=True)
```

## 3. AuditLog (tamper-proof records):

```
class AuditLog(models.Model):
 # IMMUTABLE – once created, cannot be changed or deleted
 performed_by_id = models.IntegerField() # Who did it (Integer, not
ForeignKey!)
 performed_by_name = models.CharField() # Their name at that time
 action = models.CharField() # CREATE, UPDATE, DELETE, LOGIN,
LOGOUT
 model_name = models.CharField() # "Asset", "User", "OrgUnit"
 object_id = models.CharField() # Which record was affected
 old_value = models.JSONField(null=True) # What it WAS before
 new_value = models.JSONField(null=True) # What it IS now
 timestamp = models.DateTimeField(auto_now_add=True)
 ip_address = models.CharField()
```

```
def save(self, *args, **kwargs):
 if self.pk: # If this record ALREADY exists
 raise Exception("AuditLog cannot be modified!")
 super().save(*args, **kwargs)

def delete(self, *args, **kwargs):
 raise Exception("AuditLog cannot be deleted!")
```

**Why IntegerField for user ID instead of ForeignKey?** Because the users table is in the PUBLIC schema and the audit log is in each MINISTRY'S schema. PostgreSQL cannot create a foreign key across schemas.

#### organizations/views.py — Org web pages (403 lines)

View	What it does
org_unit_list_view	Shows org tree (Ministry → Agency → Facility)
org_unit_create_view	Creates Agency or Facility under a parent
org_unit_edit_view	Edits org unit name/code
org_unit_delete_view	Deletes org unit (cannot delete if has children)
audit_log_view	Shows audit trail with action filter

#### organizations/master\_data\_views.py — Dropdown management (589 lines)

View	What it does
master_data_list_view	Lists all dropdown options grouped by category
master_data_create_view	Adds new dropdown option
master_data_edit_view	Edits dropdown option
master_data_delete_view	Deletes dropdown option
master_data_seed_view	Loads default options (funding sources, methods, etc.)
asset_category_list_view	Lists asset categories
asset_category_create_view	Creates asset category
asset_category_edit_view	Edits asset category

#### organizations/api\_views.py — Org + dashboard API (424 lines)

Class	URL	What it does
OrgUnitListAPIView	GET /api/org-units/	Returns org tree + flat facilities list
AuditLogListAPIView	GET /api/audit-logs/	Paginated audit trail with filters



Class	URL	What it does
DashboardStatsAPIView	GET /api/dashboard/stats/	Dashboard statistics

## FOLDER 5: `tenants/` — Multi-tenancy management

### `tenants/models.py` — Ministry and Domain (47 lines)

**Purpose:** Defines what a "ministry" (tenant) is and how domains map to ministries.

```
class Ministry(TenantMixin):
 # TenantMixin gives us: schema_name (e.g., "moh_schema")
 # auto_create_schema = True means when we save a Ministry,
 # PostgreSQL creates a NEW SCHEMA automatically

 name = models.CharField() # "Ministry of Health"
 short_name = models.CharField() # "MOH"
 is_active = models.BooleanField(default=True)

class Domain(DomainMixin):
 domain = models.CharField() # "moh.localhost" or "moh.platform.go.tz"
 is_primary = models.BooleanField(default=True)
 tenant = models.ForeignKey(Ministry) # This domain belongs to MOH
```

**The magic of TenantMixin:** When you create a `Ministry(name="Health")`, django-tenants automatically:

1. Runs `CREATE SCHEMA moh_schema` in PostgreSQL
2. Creates ALL tenant tables inside it (assets\_asset, organizations\_orgunit, etc.)
3. Sets up the domain mapping

### `tenants/views.py` — Ministry management (261 lines)

View	What it does	Who can access
<code>ministry_list_view</code>	Lists all ministries with stats	SUPER_ADMIN
<code>ministry_create_view</code>	Creates new ministry + schema + domain + root org unit	SUPER_ADMIN
<code>ministry_detail_view</code>	Shows ministry details: users, org units, recent assets	SUPER_ADMIN
<code>ministry_toggle_active_view</code>	Activate/deactivate a ministry	SUPER_ADMIN

**Key code — onboarding a new ministry:**

```
def ministry_create_view(request):
 if request.method == 'POST':
```

```
Step 1: Create Ministry (this auto-creates the PostgreSQL schema!)
ministry = Ministry.objects.create(
 name=form.cleaned_data['name'],
 schema_name=form.cleaned_data['schema_name'], # e.g., "moh_schema"
)
Step 2: Create domain mapping
Domain.objects.create(
 domain=form.cleaned_data['domain'], # e.g., "moh.localhost"
 tenant=ministry,
 is_primary=True,
)
Step 3: Create root org unit
with schema_context(ministry.schema_name):
 OrgUnit.objects.create(
 name=ministry.name,
 unit_type='MINISTRY',
)
```

---

## FOLDER 6: `templates/` — All HTML pages (26 files)

### `templates/shared/base.html`

**Purpose:** The MAIN LAYOUT that every page extends. Contains:

- Left sidebar with navigation links
- Top bar with search and user menu
- Message area for success/error messages
- Footer with user info

All other templates just fill in the content area. They "extend" `base.html`:

```
{% extends 'shared/base.html' %}
{% block content %}
 <!-- Page-specific content goes here -->
{% endblock %}
```

### `templates/authentication/login.html`

**Purpose:** Two-panel login page. Left side: branding. Right side: "Sign in with Government SSO" button.

### `templates/authentication/user_list.html`

**Purpose:** User table with avatar, name, email, role badge, status, actions.

### `templates/authentication/user_form.html` and `user_edit.html`

**Purpose:** Create/edit user forms with role and ministry selection.

`templates/authentication/pending_access_list.html` and `pending_access_review.html`

**Purpose:** List and approve/reject pending access requests.

`templates/dashboard/dashboard.html`

**Purpose:** Main dashboard with stat cards and expiry warning tables.

`templates/assets/asset_list.html`

**Purpose:** Asset register table with search, filters, pagination, and action buttons.

`templates/assets/asset_form.html`

**Purpose:** Create/edit asset form (two-column layout).

`templates/assets/asset_detail.html`

**Purpose:** Full asset detail view with all fields displayed.

`templates/tenants/ministry_list.html`, `ministry_form.html`, `ministry_detail.html`

**Purpose:** Ministry management pages (list, create, detail).

`templates/organizations/` (8 files)

**Purpose:** Org unit management, master data management, audit log viewer, asset category management.

`templates/shared/pagination.html`

**Purpose:** Reusable pagination component (Previous, Page 1, Page 2, Next...). Included by any page that has lists.

---

## FOLDER 7: `static/` — CSS and images

`static/css/style.css` (758 lines)

**Purpose:** ALL styling for the entire application. One file controls everything visual.

### Key sections:

```
Lines 4-58: CSS variables (colors, sizes, shadows) – THE DESIGN SYSTEM
Lines 73-212: Sidebar layout and navigation
Lines 214-259: Main content area layout
Lines 261-283: Card styling
Lines 285-331: Dashboard stat cards
Lines 333-371: Table styling
Lines 373-403: Badges (status labels)
Lines 430-469: Buttons
```

Lines 471-539: Forms and inputs  
 Lines 541-557: Alert messages  
 Lines 657-758: Login page styling

ROOT FILE: `manage.py`

**Purpose:** The ENTRY POINT for EVERYTHING. Every command goes through this file.

```
#!/usr/bin/env python
import os
import sys

def main():
 os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
 # ↑ Tells Django where to find settings
 try:
 from django.core.management import execute_from_command_line
 except ImportError:
 raise ImportError("Django is not installed.")
 execute_from_command_line(sys.argv)

if __name__ == '__main__':
 main()
```

**Commands you pass to it:**

```
python manage.py runserver # Start development server
python manage.py migrate # Create/update database tables
python manage.py migrate_schemas # Create/update tables in ALL schemas
python manage.py createsuperuser # Create admin account
python manage.py setup_demo_data # Seed demo data for presentation
python manage.py shell # Open Python command line
python manage.py test # Run tests
```

Summary: What to remember

If they ask about...	Go to this file/folder
Login process	<code>authentication/views.py</code> and <code>oidc_backend.py</code>
User accounts	<code>authentication/models.py</code> (CustomUser)
User management	<code>authentication/user_views.py</code> and <code>keycloak_admin.py</code>
API authentication	<code>authentication/api_views.py</code>

If they ask about...	Go to this file/folder
JWT tokens	<a href="#">authentication/api_serializers.py</a> and <a href="#">config/settings.py</a> (SIMPLE_JWT)
Permissions (API)	<a href="#">authentication/api_permissions.py</a>
Permissions (web)	<a href="#">authentication/decorators.py</a>
Assets (web)	<a href="#">assets/views.py</a>
Assets (API)	<a href="#">assets/api_views.py</a>
Assets (database)	<a href="#">assets/models.py</a>
Organization units	<a href="#">organizations/models.py</a> and <a href="#">views.py</a>
Audit log	<a href="#">organizations/models.py</a> (AuditLog)
Master data/dropdowns	<a href="#">organizations/master_data_views.py</a>
Multi-tenancy	<a href="#">tenants/models.py</a>
Onboard a ministry	<a href="#">tenants/views.py</a>
All settings	<a href="#">config/settings.py</a>
URL routing	<a href="#">config/urls.py</a>
API URL routing	<a href="#">authentication/api_urls.py</a>
Swagger docs	<a href="http://localhost:8000/api/docs/">http://localhost:8000/api/docs/</a>
Login page	<a href="#">templates/authentication/login.html</a>
Dashboard	<a href="#">templates/dashboard/dashboard.html</a>
All styling	<a href="#">static/css/style.css</a>
All commands & setup	Part 25 below

## PART 25: IMPORTANT COMMANDS & SETUP REFERENCE — Everything you need to run the system

### In this part:

- [Step 0 — PostgreSQL \(do nothing\)](#)
- [Step 1 — Find your IP \(do this first, every time\)](#)
- [Step 2 — Start Keycloak](#)
- [Step 3 — Start Django](#)
- [Step 4 — Connect your phone \(do this every time the IP changes\)](#)
- [Step 5 — Load clean demo data](#)
- [Step 6 — Open everything in the browser](#)
- [Troubleshooting table](#)
- [Test accounts](#)

- [Correct startup order](#)

---

## Step 0 — PostgreSQL (Do Nothing)

PostgreSQL starts automatically when Windows boots. You never need to touch it. If you want to confirm it is running, press **Win + R**, type `services.msc`, find **postgresql-x64-17** (or similar name) — it should say **Running**. That is all.

---

## Step 1 — Find Your IP Address (Do This First, Every Time)

Open any terminal and run:

```
ipconfig
```

Look for **IPv4 Address** under your active adapter. If you are using a phone hotspot, it will look like `192.168.43.x`. Write down that number — you will use it in Steps 3 and 4.

---

## Step 2 — Start Keycloak

Open a terminal, navigate to the Keycloak bin folder, then start it:

```
cd C:\keycloak\keycloak-26.6.2\bin
kc.bat start-dev --http-port=8180
```

**Why `start-dev`:** the normal `start` command requires SSL certificates and a production setup. `start-dev` skips all that and runs Keycloak in development mode — perfect for your project and presentations.

**Why `--http-port=8180`:** Keycloak's default port is 8080, which would clash with Django or other services. Port 8180 keeps them separate.

**Wait until you see this line before continuing:**

```
Keycloak 26.6.2 on JVM started
```

Keycloak is slow — give it about 30–60 seconds. Do not close this terminal window.

---

## Step 3 — Start Django

Open a **second, separate** terminal window (keep Keycloak running in the first one):

```
cd D:\government_asset_platform
```

```
venv\Scripts\activate
```

You should see **(venv)** appear at the start of your terminal line. That means the virtual environment is active. If you accidentally use the wrong venv, nothing will work — always activate from **D:\government\_asset\_platform\venv**.

Now start Django:

```
python manage.py runserver 0.0.0.0:8000
```

**Why 0.0.0.0:** this tells Django to listen to ALL devices in the room (your phone, your laptop), not just itself. Without this, your phone cannot reach Django at all, even on the same hotspot.

**Why 8000:** this is your port number — the specific door Django listens on.

Keep this terminal open. Do not close it.

---

## Step 4 — Connect Your Phone (Do This Every Time the IP Changes)

Every time you switch WiFi networks or hotspots, your IP address changes. You must update these **four things** every single time:

---

### 4A — Update the Flutter App's IP

Open this file:

```
C:\Users\Hemed\govasset_mobile\lib\config\api_config.dart
```

Find this line and change the IP:

```
static const String serverIp = 'PUT_YOUR_NEW_IP_HERE';
```

For example, if your IP is **192.168.43.105**:

```
static const String serverIp = '192.168.43.105';
```

Save the file.

---

### 4B — Update Django's Allowed Hosts

Open this file:

```
D:\government_asset_platform\config\settings.py
```

Find **ALLOWED\_HOSTS** and add your new IP:

```
ALLOWED_HOSTS = [
 'localhost',
 '127.0.0.1',
 '.localhost',
 '192.168.43.105', # ← your new IP here
]
```

Save the file.

---

#### 4C — Register the New IP as a Domain in Django (Most Forgotten Step)

This is the step that causes the **404 error** and **"no internet connection"** on the phone if skipped. Django-tenants checks every incoming request against a registered domain list. If your new IP is not in that list, it turns the phone away before your code even runs.

Run these commands:

```
cd D:\government_asset_platform
venv\Scripts\activate
python manage.py shell
```

Once inside the shell, paste these lines exactly, replacing the IP with yours:

```
from tenants.models import Domain, Ministry
public = Ministry.objects.get(schema_name='public')
Domain.objects.get_or_create(domain='192.168.43.105', tenant=public,
is_primary=False)
print("Done")
exit()
```

You should see **Done** printed. Now exit the shell with **exit()**.

---

#### 4D — Reinstall the App on Your Phone

Connect your phone to your laptop with a USB cable. Make sure **USB debugging** is ON in your phone's developer settings. Then run:



```
cd C:\Users\Hemed\govasset_mobile
flutter run
```

This rebuilds the app with the new IP address baked in and installs it fresh on your phone. Wait until the app opens on your phone screen before unplugging the USB cable.

**Why you must rebuild:** the IP address is baked into the app at build time. Changing the file alone does nothing — the app needs to be rebuilt and reinstalled so the new IP is inside the installed version on your phone.

After `flutter run` finishes and the app opens on your phone, you can unplug the USB cable. The app will keep working over WiFi/hotspot.

---

## Step 5 — Load Clean Demo Data (Optional but Recommended Before Presentations)

```
cd D:\government_asset_platform
venv\Scripts\activate
python manage.py setup_demo_data
```

This clears leftover test clutter and loads clean, professional-looking demo data for both ministries. Run this before any presentation or demonstration.

---

## Step 6 — Open Everything in the Browser

Once Django and Keycloak are both running:

What	Address
Web platform	<a href="http://localhost:8000">http://localhost:8000</a>
API documentation (Swagger)	<a href="http://localhost:8000/api/docs/">http://localhost:8000/api/docs/</a>
Keycloak admin panel	<a href="http://localhost:8180">http://localhost:8180</a>
Django admin panel	<a href="http://localhost:8000/admin/">http://localhost:8000/admin/</a>

To test from your phone's browser on the same hotspot, replace `localhost` with your IP:

```
http://192.168.43.105:8000
```

---

## Quick Troubleshooting Reference

Problem you see	Most likely cause	Fix
Phone shows "No internet connection"	IP in <code>api_config.dart</code> is old	Update <code>serverIp</code> and run <code>flutter run</code> again
Phone gets a 404 error	New IP not registered as Domain	Run Step 4C again with new IP
Web login does nothing or errors	Keycloak not started	Run Step 2, wait for it to fully load
<code>ModuleNotFoundError</code> when starting Django	Wrong virtual environment	Run <code>venv\Scripts\activate</code> from <code>D:\government_asset_platform</code>
Django says <code>ALLOWED_HOSTS</code> error	New IP not in <code>settings.py</code>	Run Step 4B
Phone app shows old data after IP change	App not rebuilt	Reconnect USB and run <code>flutter run</code> again
Keycloak page not loading	Still starting up	Wait 60 seconds, try again

Test Accounts (All Use Password: `Admin@123`)

Username	Role	Ministry
<code>superadmin</code>	Super Admin	Platform-wide
<code>moh_admin</code>	Ministry Admin	Ministry of Health
<code>mnh_manager</code>	Agency Manager	Ministry of Health
<code>rad_clerk</code>	Facility Clerk	Ministry of Health
<code>moh_auditor</code>	Auditor	Ministry of Health
<code>mof_admin</code>	Ministry Admin	Ministry of Finance

The Correct Startup Order (Every Time)

1. Check IP (`ipconfig`)
2. Start Keycloak (wait for it to fully load)
3. Start Django (`python manage.py runserver 0.0.0.0:8000`)
4. If IP changed → update `api_config.dart` + `settings.py` + register domain + `flutter run`
5. Test web login in browser
6. Test mobile login on phone
7. Run `setup_demo_data` if doing a presentation

Print or screenshot this page and keep it next to your laptop.